دانشگاه علم و صنعت ایران

**دانشکده مهندسی برق**

# پروژه پایانی مدار منطقی پیشرفته

# طراحی پردازنده MIPS


**نگارش**

**محمد رضا کریمیان**

**استاد درس**

**دکتر ستار میرزاکوچکی**

**شماره دانشجویی**

**٤٠٢٤١٢٣٧٢**


**بهار ١٤٠٥**

# فهرست مطالب

صفحه

**۱- مقدمه ..... ۷**
۱-۱- مقدمه پردازنده MIPS ..... ۷
۱-۲- هدف پروژه ..... ۷
۱-۳- نحوه پیاده سازی پروژه ..... ۹
**۲- مبانی طراحی سخت افزار پردازنده ..... ۱۰**
۲-۱- مدار های ترکیبی ..... ۱۰
۲-۲- مدار های ترتیبی و عناصر حالت ..... ۱۱
۲-۳- روش زمان بندی کلاک ..... ۱۲
۲-۴- ریجسترها ..... ۱۳
۲-۴-۱- Register File ..... ۱۳
۲-۵- Instruction Memory ..... ۱۴
۲-۶- Data Memory ..... ۱۵
۲-۷- باس داده (Bus) ..... ۱۵
**۳- معماری پردازنده طراحی شده ..... ۱۶**
۳-۱- مسیر داده (Datapath) ..... ۱۷
۳-۲- واحد کنترلی (Control Unit) ..... ۱۸
۳-۳- نحوه اجرای دستورات ..... ۱۹
۳-۳-۱- مرحله Fetch ..... ۲۰
۳-۳-۲- مرحلهِ Decode ..... ۲۰
۳-۳-۳- مرحله Execute ..... ۲۰
۳-۳-۴- مرحله Memory Access ..... ۲۰
۳-۳-۵- مرحله Write Back ..... ۲۱
**۴- طراحی و پیاده سازی اجزای پردازنده (ماژول ها) در Verilog ..... ۲۱**
۴-۱- Program Counter (PC) ..... ۲۲
۴-۲- Instruction Memory ..... ۲۳
۴-۳- Register File ..... ۲۵
۴-۴- Sign Extend ..... ۲۷
۴-۵- MUX ..... ۲۹
۴-۶- ALU Control ..... ۳۰
۴-۷- ALU ..... ۳۲
۴-۸- Data Memory ..... ۳۴
۴-۹- Adder ..... ۳۶

۴-۱۰- shift left 2 ..... ۳۷
۴-۱۱- Datapath ..... ۳۸
۴-۱۲- Control Unit ..... ۴۱
۴-۱۳- CPU ..... ۴۲

**۵- تحلیل و ارزیابی مسیر اجرای هر دستور ..... ۴۳**
۵-۱- مسیر اجرای دستورات R-type ..... ۴۳
۵-۲- مسیر اجرای دستورات Immediate ..... ۴۵
۵-۳- مسیر اجرای دستورات LW و SW ..... ۴۶
۵-۴- مسیر اجرای دستورات Branch ..... ۴۷
۵-۵- مسیر اجرای دستورات Jump ..... ۴۹

**۶- ارزیابی عملکرد پردازنده با استفاده از شبیه سازی (Testbench) ..... ۵۱**
۶-۱- طراحی Testbench ..... ۵۱
۶-۲- شبیه سازی ..... ۵۲
۶-۲-۱- بررسی دستور اول (دستور LW) ..... ۵۳
۶-۲-۲- بررسی دستور دوم (دستور ADDI) ..... ۵۴
۶-۲-۳- بررسی دستور سوم (دستور ANDI) ..... ۵۴
۶-۲-۴- بررسی دستورچهارم (دستور ORI) ..... ۵۵
۶-۲-۵- بررسی دستور پنجم (دستور ADD R-type) ..... ۵۵
۶-۲-۶- بررسی دستور ششم (دستور SUB R-type) ..... ۵۶
۶-۲-۷- بررسی دستور هفتم (دستور AND R-type) ..... ۵۷
۶-۲-۸- بررسی دستور هشتم (دستور OR R-type) ..... ۵۷
۶-۲-۹- بررسی دستور نهم (دستور SLT R-type) ..... ۵۸
۶-۲-۱۰- بررسی دستور دهم (دستور SW) ..... ۵۹
۶-۲-۱۱- بررسی دستور یازدهم (دستور LW) ..... ۵۹
۶-۲-۱۲- بررسی دستور دوازدهم (دستور BEQ) ..... ۶۰
۶-۲-۱۳- بررسی دستور سیزدهم (دستور BNQ) ..... ۶۱
۶-۲-۱۴- بررسی دستورات چهاردهم تا شانزدهم ..... ۶۱
۶-۲-۱۵- بررسی دستور هفدهم (دستور Jump) ..... ۶۱

**۷- جمع بندی و چالش های طول پروژه ..... ۶۲**

# فهرست اشکال

صفحه


۱-۱- نمای کلی پردازنده طراحی شده ..... ۹
۱-۲- یک مرور کلی بر پیاده سازی سخت افزاری زیرمجموعه ای از دستورات پردازنده ..... ۱۰
۲-۲- تفاوت مدار ترکیبی و ترتیبی ..... ۱۲
۲-۳- مدار های ترکیبی، عناصر حالت و کلاک ، ارتباط نزدیکی دارند ..... ۱۲
۲-۴- بلوک دیاگرام کلی یک رجیستر ..... ۱۳
۲-۵- بلوک دیاگرام کلی یک رجیستر فایل ..... ۱۴
۲-۶- بلوک دیاگرام کلی یک Instruction Memory ..... ۱۴
۲-۷- بلوک دیاگرام کلی یک Data Memory ..... ۱۵
۲-۸- مفهوم Data Bus ..... ۱۶
۳-۱- ارتباط مسیر داده (data path) و واحد کنترلی (control unit) ..... ۱۷
۳-۲- واحد کنترلی (control unit) ..... ۱۹
۳-۳- پنج گام مفهومی اجرای یک دستور ..... ۲۰
۴-۱- ماژول PC ..... ۲۲
۴-۲- ماژول Instruction Memory ..... ۲۴
۴-۳- ماژول Register File ..... ۲۵
۴-۴- ماژول Sign extend ..... ۲۷
۴-۵- ماژول MUX1 ..... ۲۸
۴-۶- ماژول MUX2 ..... ۲۹
۴-۷- ماژول ALU Control ..... ۳۰
۴-۸- ماژول ALU ..... ۳۳
۴-۹- ماژول Data Memory ..... ۳۵
۴-۱۰- ماژول Adder ..... ۳۷
۴-۱۱- ماژول Shift Left2 ..... ۳۸
۴-۱۲- ماژول CPU ..... ۴۲
۶-۱- دستورات قرار داده شده در فایل instrmem_text ..... ۵۲
۶-۲- شکل موج های خروجی PC، Clock، Reset، Register file، mem_array[101] ..... ۵۳

# فهرست جداول

صفحه

۱-۱- نمای کلی از دستورات پشتیبانی شده..... ۸
۱-۲- مدار های ترکیبی پروژه..... ۱۱
۲-۲- عناصر حالت استفاده شده در پروژه..... ۱۱
۱-۳- سیگنال های کنترلی برای هر دستور..... ۱۹
۱-۴- مشخصات سیگنال های ماژول Register File..... ۲۶
۲-۴- نگاشت ALUop و Function به عملیات ALU..... ۳۲
۱-۵- سیگنال های کنترلی R-type..... ۴۴
۲-۵- سیگنال های کنترلی Immediate..... ۴۵
۳-۵- سیگنال های کنترلی LW..... ۴۶
۴-۵- سیگنال های کنترلی SW..... ۴۶
۵-۵- سیگنال های کنترلی BEQ..... ۴۸
۶-۵- سیگنال های کنترلی BNQ..... ۴۸
۷-۵- سیگنال های کنترلی Jump..... ۵۰

# فهرست کد ها

صفحه


۱-۴- ماژول PC ..... ۲۲
۲-۴- ماژول Instruction Memory ..... ۲۴
۳-۴- ماژول Register File ..... ۲۶
۴-۴- ماژول Sign Extend ..... ۲۸
۵-۴- ماژول MUX1 ..... ۲۹
۶-۴- ماژول MUX2 ..... ۲۹
۷-۴- ماژول ALU Control ..... ۳۱
۸-۴- ماژول ALU ..... ۳۳
۹-۴- ماژول Data Memory ..... ۳۵
۱۰-۴- ماژول Adder ..... ۳۷
۱۱-۴- ماژول Shift Left2 ..... ۳۸
۱۲-۴- ماژول datapath ..... ۳۹
۱۳-۴- ماژول Control Unit ..... ۴۱
۱۴-۴- ماژول CPU ..... ۴۲
۱-۶- Testbench ..... ۵۱

# مقدمه:

## ۱-۱- معرفی پردازنده MIPS:

پردازنده MIPS یکی از شناخته‌شده‌ترین معماری‌های مبتنی بر RISC (Reduced Instruction Set Computer) است که به دلیل سادگی، نظم در قالب دستورها و سهولت پیاده‌سازی، به طور گسترده مورد استفاده قرار می‌گیرد. در این معماری، تمامی دستورها دارای قالب مشخص بوده و اغلب عملیات در قالب چند گروه اصلی شامل دستورات محاسباتی، دستورات دسترسی به حافظه و دستورات پرشی انجام می‌شود. همین ویژگی موجب شده است که طراحی مسیر داده (Datapath) و واحد کنترل (Control Unit) با پیچیدگی کمتری نسبت به بسیاری از معماری‌های دیگر انجام گیرد. فرآیند طراحی در دو مرحله اصلی انجام می شود. مسیر داده وظیفه انتقال اطلاعات بین واحدهای مختلف پردازنده را بر عهده دارد، در حالی که واحد کنترل با تولید سیگنال‌های کنترلی مناسب، عملکرد هر بخش را مطابق دستور در حال اجرا تنظیم می‌کند. همکاری صحیح این دو بخش موجب می‌شود دستورهای مختلف بدون تداخل و با ترتیب صحیح اجرا شوند. در معماری MIPS ، عملیات پردازش داده‌ها عمدتاً بر روی رجیسترها انجام می‌شود و دسترسی مستقیم به حافظه تنها توسط دستورهای Load و Store امکان‌پذیر است. این ویژگی موجب کاهش زمان اجرای دستورها و افزایش کارایی پردازنده می‌شود. علاوه بر این، وجود قالب‌های ثابت برای دستورها فرآیند رمزگشایی دستور (Instruction Decode) را ساده‌تر کرده و طراحی سخت‌افزار را تسهیل می‌کند. در این پروژه یک پردازنده تک‌سیکل (Single Cycle Processor) مبتنی بر معماریMIPS طراحی و با استفاده از زبان توصیف سخت‌افزار Verilog پیاده‌سازی شده است. در این پردازنده، هر دستور تنها در یک سیکل کلاک اجرا می‌شود و تمام مراحل واکشی، رمزگشایی، اجرا، دسترسی به حافظه و نوشتن نتیجه در همان سیکل انجام می‌گیرد. اگرچه این روش موجب ساده شدن طراحی می‌شود، اما به دلیل وابستگی زمان کلاک به طولانی‌ترین مسیر داده، از نظر سرعت نسبت به معماری‌های چندمرحله‌ای و پایپلاین دارای محدودیت است.

## ۲-۱- هدف پروژه :

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک پردازنده Single Cycle MIPS با استفاده از زبان توصیف سخت‌افزار Verilog است. در این معماری، هر دستور در طول یک سیکل کلاک اجرا شده و تمامی مراحل اجرای دستور شامل واکشی، رمزگشایی، اجرا، دسترسی به حافظه و نوشتن نتیجه در همان سیکل انجام می‌شود. اگرچه این روش باعث افزایش طول سیکل

کلاک نسبت به معماری‌های پیشرفته‌تر می‌شود، اما به دلیل سادگی ساختار، یکی از مناسب‌ترین روش‌ها برای آموزش مفاهیم طراحی پردازنده محسوب می‌شود.

در این پروژه تمامی اجزای اصلی پردازنده شامل شمارنده برنامه (PC) ، حافظه دستور، فایل رجیسترها، واحد محاسبات و منطق (ALU) ، حافظه داده، واحد توسعه علامت، مالتی‌پلکسرها، جمع‌کننده‌ها، مسیر داده و واحد کنترل به صورت ماژول‌های مستقل طراحی شده و سپس در قالب یک پردازنده کامل به یکدیگر متصل شده‌اند.

علاوه بر دستورهای پایه معماری MIPS ، در این پروژه دستورهای Immediate شامل ADDI ، ANDI، ORI و SLTI نیز پیاده‌سازی شده‌اند.

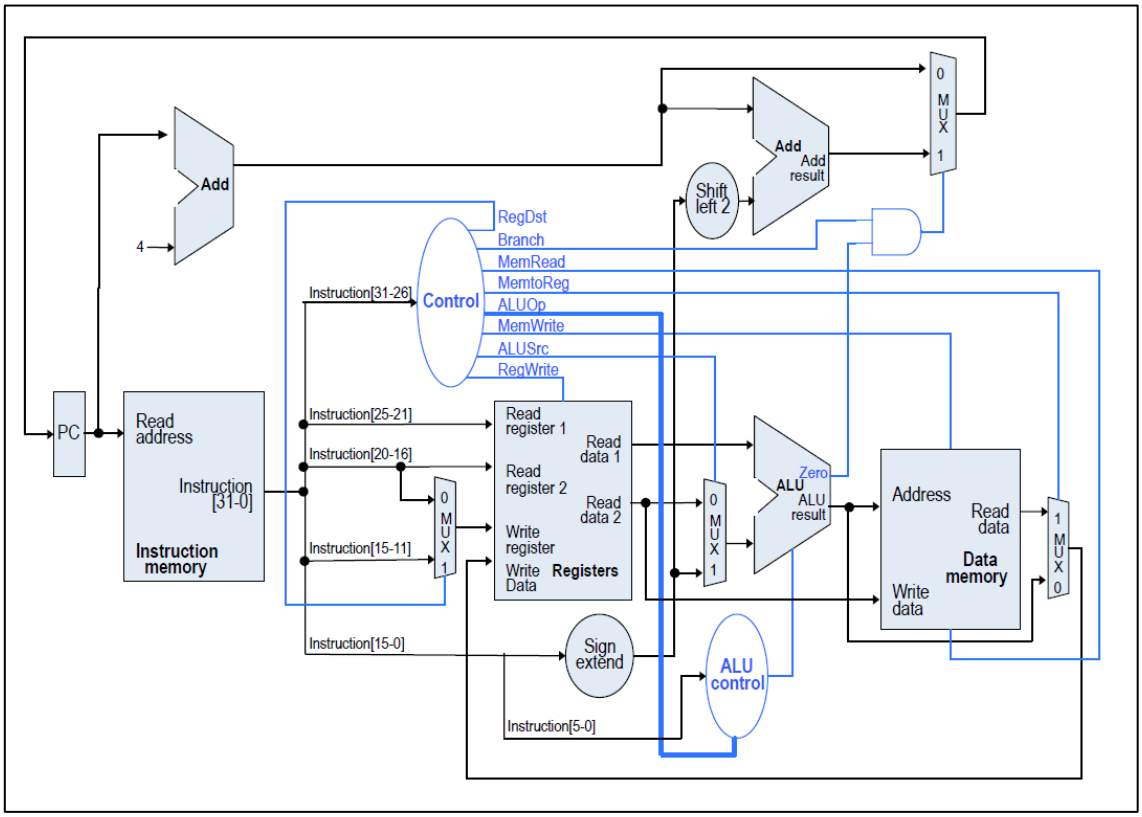
| نام دستور | Opcode | کلاس دستور | توضیح |
|---|---|---|---|
| R-type (add,sub,and,or,slt) | 000000 | محاسباتی-منطقی | عملیات بین دو رجیستر |
| lw | 100011 | مراجعه به حافظه | بارگذاری از حافظه به رجیستر |
| sw | 101011 | مراجعه به حافظه | ذخیره‌ی محتوای رجیستر در حافظه |
| beq | 000100 | پرش شرطی | پرش در صورت برابری |
| bnq (BNE) | 000101 | پرش شرطی | پرش در صورت نابرابری |
| j (Jump) | 000010 | پرش غیرشرطی | پرش مستقیم به آدرس مقصد |
| addi | 001000 | Immediate | جمع با یک مقدار ثابت ۱۶ بیتی |
| andi | 001100 | Immediate | AND منطقی با مقدار ثابت |
| ori | 001101 | Immediate | OR منطقی با مقدار ثابت |
| slti | 001010 | Immediate | مقایسه‌ی علامت‌دار با مقدار ثابت |

جدول ۱-۱: نمای کلی از دستور های پشتیبانی شده

## ۳-۱- نحوه پیاده سازی پروژه :

در این پروژه، فرآیند طراحی به صورت ماژولار انجام شده است؛ بدین معنا که هر بخش از پردازنده ابتدا به صورت یک ماژول مستقل در زبان Verilog طراحی و شبیه‌سازی شده و پس از اطمینان از عملکرد صحیح، تمامی ماژول‌ها در قالب مسیر داده اصلی به یکدیگر متصل شده‌اند. این روش طراحی علاوه بر افزایش خوانایی کد، امکان توسعه و رفع اشکال هر بخش را نیز به شکل قابل توجهی ساده‌تر می‌کند.

واحد کنترل بر اساس Opcode هر دستور، سیگنال‌های کنترلی لازم برای انتخاب مسیر داده، تعیین عملیات ALU ، کنترل حافظه و نوشتن در رجیسترها را تولید می‌کند. در ادامه، مسیر داده با استفاده از این سیگنال‌ها عملیات مورد نیاز را برای اجرای دستور انجام می‌دهد.

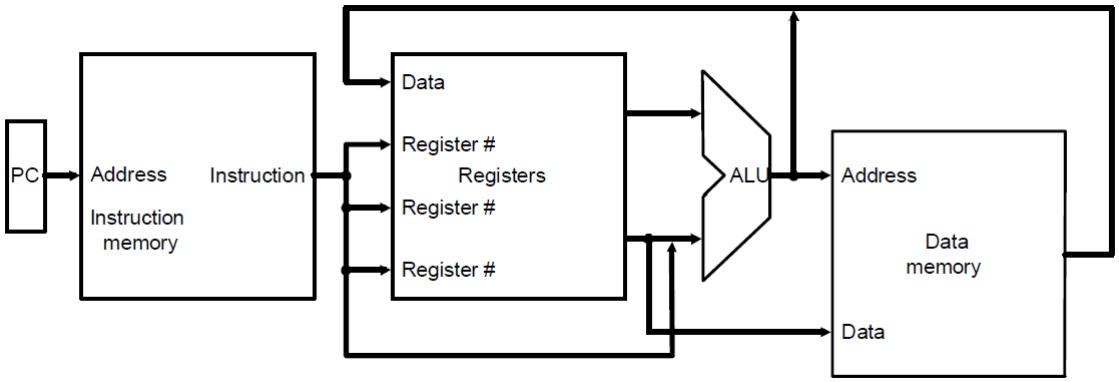



شکل ۱-۱: نمای کلی پردازنده طراحی شده

## ۲- مبانی طراحی سخت‌افزار پردازنده :

طراحی یک پردازنده تنها به نوشتن کدهای سخت‌افزاری محدود نمی‌شود، بلکه نیازمند شناخت دقیق اجزای تشکیل‌دهنده سیستم دیجیتال و نحوه ارتباط آن‌ها با یکدیگر است. هر پردازنده از مجموعه‌ای از واحدهای محاسباتی، حافظه‌ها، رجیسترها، مسیرهای انتقال داده و واحدهای کنترلی تشکیل شده است که هماهنگی میان آن‌ها توسط سیگنال کلاک و منطق کنترل انجام می‌شود.

در معماری MIPS ، به دلیل سادگی مجموعه دستورها و یکنواخت بودن قالب آن‌ها، بسیاری از واحدهای سخت‌افزاری بین دستورهای مختلف مشترک هستند. به همین دلیل، درک مفاهیمی مانند مدارهای ترکیبی، عناصر حالت، رجیسترها و روش زمان‌بندی کلاک، پیش از بررسی ساختار پردازنده ضروری است.




شکل ۱-۲: یک مرور کلی بر پیاده سازی سخت افزاری زیر مجموعه ای از دستورات پردازنده MIPS که در آن فقط ماژول های عملیاتی اصلی و ارتباطات اصلی نشان داده شده است

### ۱-۲- مدار های ترکیبی :

مدارهای ترکیبی مدارهایی هستند که خروجی آن‌ها تنها به مقدار لحظه‌ای ورودی‌ها وابسته است و هیچ‌گونه حافظه داخلی برای نگهداری اطلاعات ندارند. بنابراین، اگر ورودی‌های یک مدار ترکیبی ثابت باقی بمانند، خروجی آن نیز همواره مقدار ثابتی خواهد داشت.

در پردازنده MIPS بسیاری از واحدهای محاسباتی از نوع مدارهای ترکیبی هستند. این واحدها تنها عملیات موردنظر را روی داده‌های ورودی انجام داده و نتیجه را بدون ذخیره‌سازی داخلی در خروجی قرار می‌دهند. از جمله مهم‌ترین مدارهای ترکیبی در این پروژه می‌توان به واحد محاسبات و منطق (ALU) ، مالتی‌پلکسرها (MUX) ، جمع‌کننده‌ها(Adder) ، واحد توسعه علامت (Sign Extend) و شیفت‌دهنده دو بیتی (Shift Left 2) اشاره کرد.

استفاده از مدارهای ترکیبی باعث می‌شود عملیات پردازشی با حداقل تأخیر انجام شود و هر تغییر در ورودی‌ها، بلافاصله در خروجی منعکس گردد.

| ماژول | وظیفه ماژول |
|---|---|
| ALU | انجام عملیات محاسباتی و منطقی |
| MUX | انتخاب یکی از چند ورودی |
| ADD | جمع دو مقدار |
| Sign Extend | توسعه بیت علامت |
| Shift Left | شیفت دو بیتی داده |

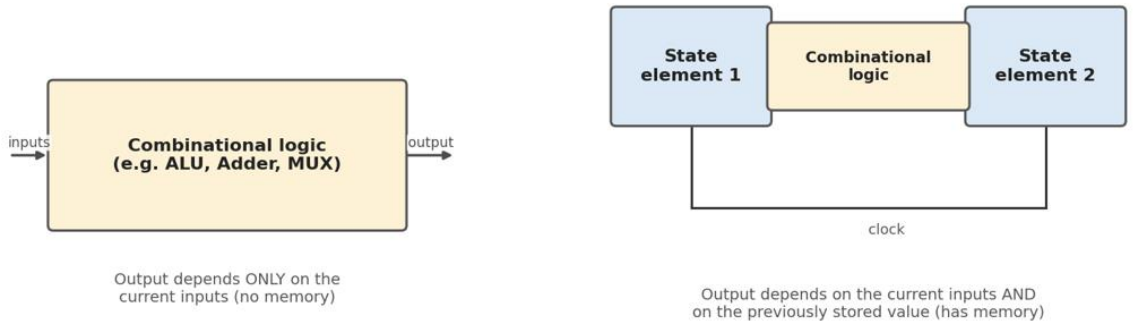
جدول ۱-۲: مدار های ترکیبی پروژه

## ۲-۲- مدار های ترتیبی و عناصر حالت :

برخلاف مدارهای ترکیبی، عناصر حالت قادر به ذخیره اطلاعات هستند و خروجی آن‌ها علاوه بر ورودی‌های فعلی، به اطلاعات ذخیره‌شده در گذشته نیز وابسته است. این عناصر نقش حافظه را در پردازنده ایفا می‌کنند و وضعیت جاری سیستم را در طول اجرای برنامه حفظ می‌کنند. این واحدها اطلاعات مورد نیاز پردازنده را بین سیکل‌های کلاک نگهداری کرده و امکان اجرای صحیح برنامه را فراهم می‌کنند.

| ماژول | وظیفه ماژول |
|---|---|
| PC | نگهداری آدرس دستور جاری |
| Register File | نگهداری داده‌های پردازشی |
| Intruction Memory | ذخیره دستورها |
| Data Memory | ذخیره داده‌ها |

جدول ۲-۲: عناصر حالت استفاده شده در پروژه



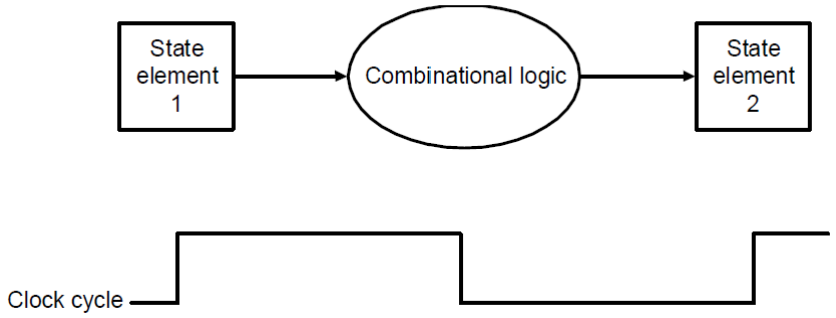

شکل ۲-۲: تفاوت مدار ترکیبی و ترتیبی

## ۳-۲- روش زمان بندی کلاک :

در سیستم‌های دیجیتال، هماهنگی میان واحدهای مختلف توسط سیگنال کلاک انجام می‌شود. روش زمان‌بندی کلاک مشخص می‌کند که اطلاعات در چه زمانی خوانده شده و در چه زمانی در عناصر حافظه ذخیره شوند. اگر این زمان‌بندی به درستی تعریف نشود، ممکن است داده‌های ناپایدار یا اشتباه در مدار منتقل شوند و عملکرد پردازنده دچار خطا گردد.

در این پروژه از کلاک حساس به لبه استفاده شده است. در این روش، داده‌ها تنها در لبه بالارونده سیگنال کلاک در عناصر حالت ثبت می‌شوند. این روش باعث می‌شود تمام واحدهای ترتیبی پردازنده به صورت هماهنگ عمل کرده و از بروز مشکلات ناشی از خواندن و نوشتن همزمان جلوگیری شود.

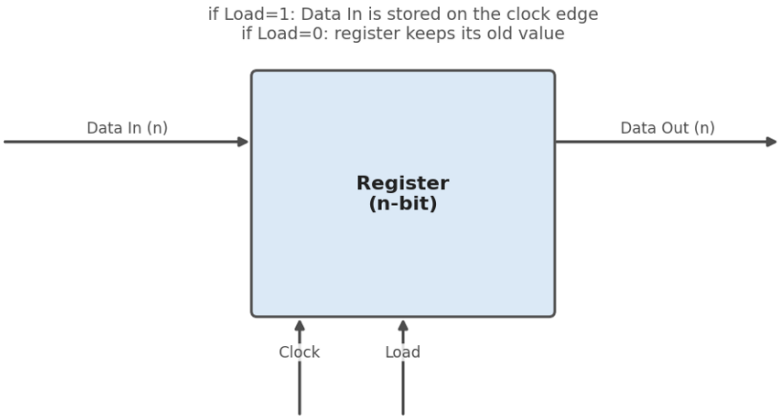
به همین دلیل در ماژول‌هایی مانند PC ، Register File و Data Memory عملیات نوشتن اطلاعات در لبه مثبت کلاک انجام می‌شود، در حالی که بسیاری از واحدهای ترکیبی بدون نیاز به کلاک و به صورت لحظه‌ای خروجی خود را تولید می‌کنند.




شکل ۳-۲: مدارهای ترکیبی، عناصر حالت و کلاک، ارتباط نزدیکی به هم دارند

## ۴-۲- ریجستر ها :

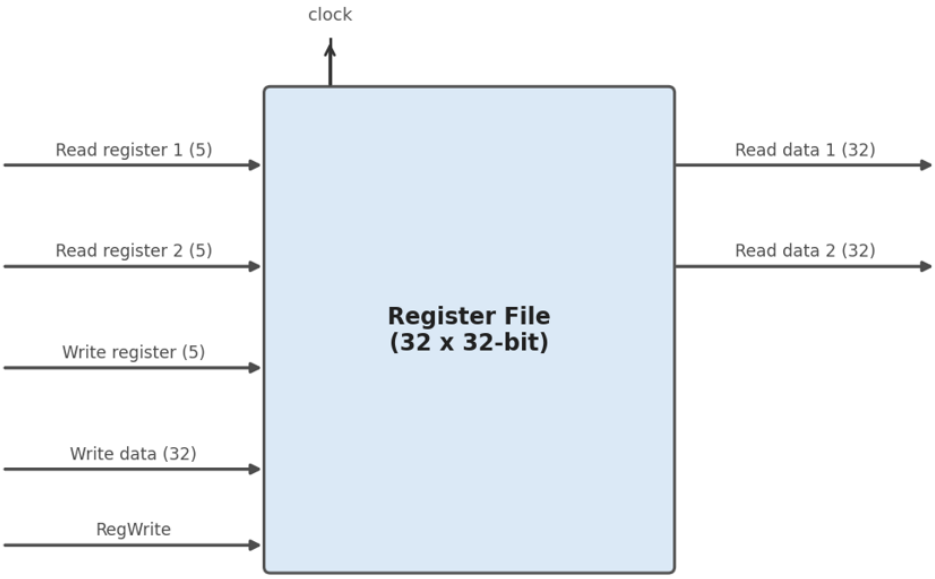
یک رجیستر، ساده‌ترین واحد ذخیره‌سازی در سطح بالاتر از فلیپ‌فلاپ منفرد است و از کنار هم قرار گرفتن چند فلیپ‌فلاپ (به تعداد بیت‌های موردنیاز) ساخته می‌شود. رجیستر معمولاً دارای دو ورودی کنترلی کلاک و Load است: هرگاه Load فعال باشد، مقدار ورودی داده در لبه‌ی کلاک ذخیره می‌شود و در غیر این صورت، رجیستر مقدار قبلی خود را حفظ می‌کند. در پروژه‌ی حاضر، رجیستر PC نمونه‌ای از این مفهوم است: در ماژول PC ، ورودی reset اولویت دارد و در صورت فعال بودن، خروجی را صفر می‌کند؛ در غیر این صورت مقدار ورودی data_in در هر لبه‌ی مثبت کلاک در data_out ذخیره می‌شود.




شکل ۴-۲: بلوک دیاگرام کلی یک رجیستر

### ۴-۲-۱- Register File :

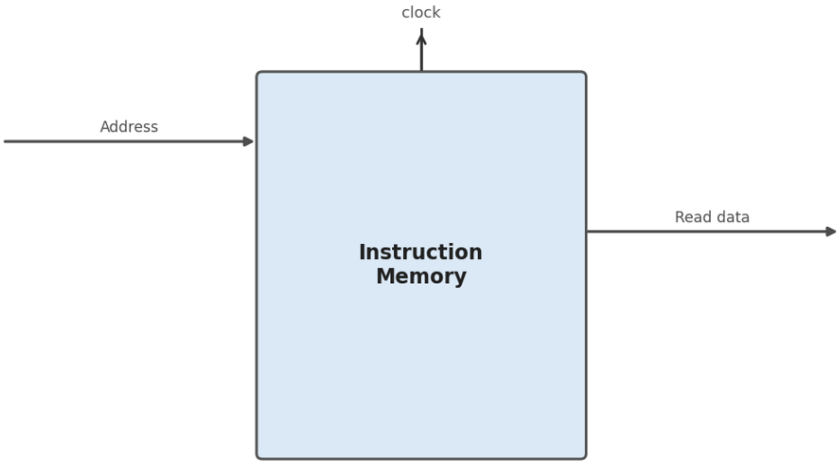
مجموعه‌ای از رجیسترهای عمومی پردازنده است که به‌جای پیاده‌سازی جداگانه، در قالب یک ماژول واحد با قابلیت آدرس‌دهی سازمان‌دهی می‌شوند .ماژول Register File این پروژه دارای ۳۲ رجیستر ۳۲ بیتی، دو پورت خواندن آدرس و دو پورت خواندن داده، دو پورت نوشتن آدرس و داده و یک پورت RegWrite که یک سیگنال کنترلی برای نوشتن در رجیسترها است، می باشد. وجود دو پورت خواندن هم‌زمان ضروری است، زیرا اغلب دستورات نظیر انواع (R-type) و (beq) به‌طور هم‌زمان به دو رجیستر برای انجام عملیات خود نیاز دارند. رجیستر شماره صفر این رجیستر فایل همواره مقدار ثابت صفر دارد.




شکل ۵-۲: بلوک دیاگرام کلی یک رجیستر فایل

## ۵-۲- Instruction Memory :

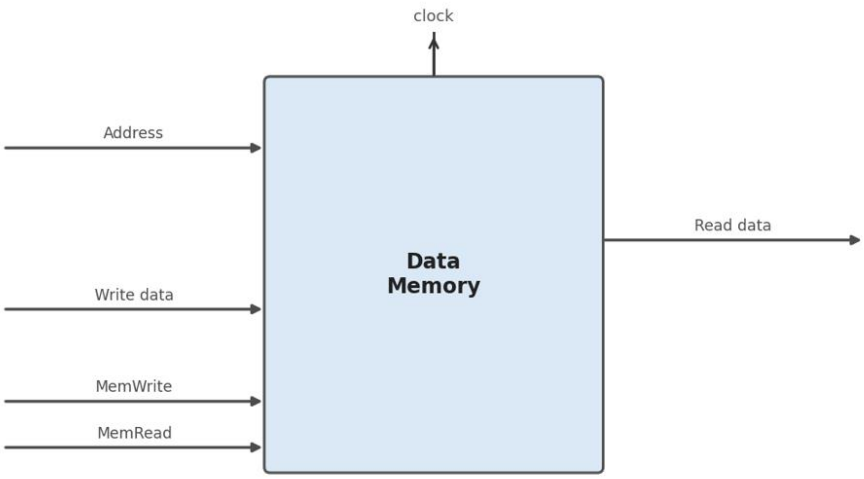
واحدی است که برنامه (مجموعه‌ای از دستورات ۳۲ بیتی) را نگه می‌دارد و بر اساس آدرس دریافتی از PC دستور مربوطه را در خروجی قرار می‌دهد. در این پروژه، ماژول Instruction شامل آرایه‌ای با ۱۲۸ خانه‌ی ۳۲ بیتی (mem_array[127:0]) است که محتوای آن هنگام شبیه‌سازی از فایل متنی instrmem_test.txt توسط دستور $readmemb بارگذاری می شود.




شکل ۶-۲: بلوک دیاگرام کلی یک Instruction memory

## ۶-۲- Data Memory :

بر خلاف Instruction Memory ، هم قابل خواندن و هم قابل نوشتن است و توسط دستورات sw و lw مورد استفاده قرار می گیرد. در ماژول data_memory این پروژه، آرایه‌ای مشابه با ۱۲۸ خانه‌ی ۳۲ بیتی در نظر گرفته شده و دو سیگنال کنترلی مجزا memory_read و memory_write برای انتخاب نوع عملیات وجود دارد. عملیات خواندن به‌صورت ترکیبی (تحت شرط memory_read && ~memory_write) و عملیات نوشتن هم زمان با لبه مثبت کلاک (تحت شرط ~memory_read && memory_write) انجام می شود. مقدار دهی اولیه mem_array[127]=77 هم نیز برای فراهم کردن یک مقدار قابل‌تست در فرآیند شبیه‌سازی در کد گنجانده شده است.

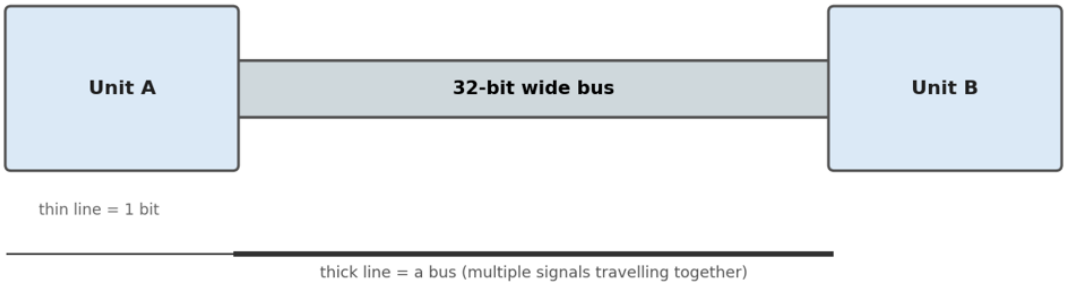



شکل ۷-۲: بلوک دیاگرام کلی یک Data memory

## ۷-۲- باس داده (Bus) :

برای برقراری ارتباط میان واحدهای مختلف پردازنده از گذرگاه‌ها (Bus) استفاده می‌شود. هر Bus مجموعه‌ای از خطوط انتقال است که داده‌های چند بیتی را میان اجزای مختلف سیستم منتقل می‌کند. در این پروژه، تقریباً تمامی Busهای داده دارای عرض ۳۲ بیت هستند تا بتوانند اطلاعات مورد نیاز معماری MIPS را منتقل کنند.

Datapath پردازنده مجموعه‌ای از این Bus ها و واحدهای پردازشی است که داده را از زمان fetch دستور تا ثبت نتیجه هدایت می‌کند. طراحی صحیح این مسیر نقش مهمی در کاهش پیچیدگی سخت‌افزار و افزایش قابلیت توسعه سیستم دارد.

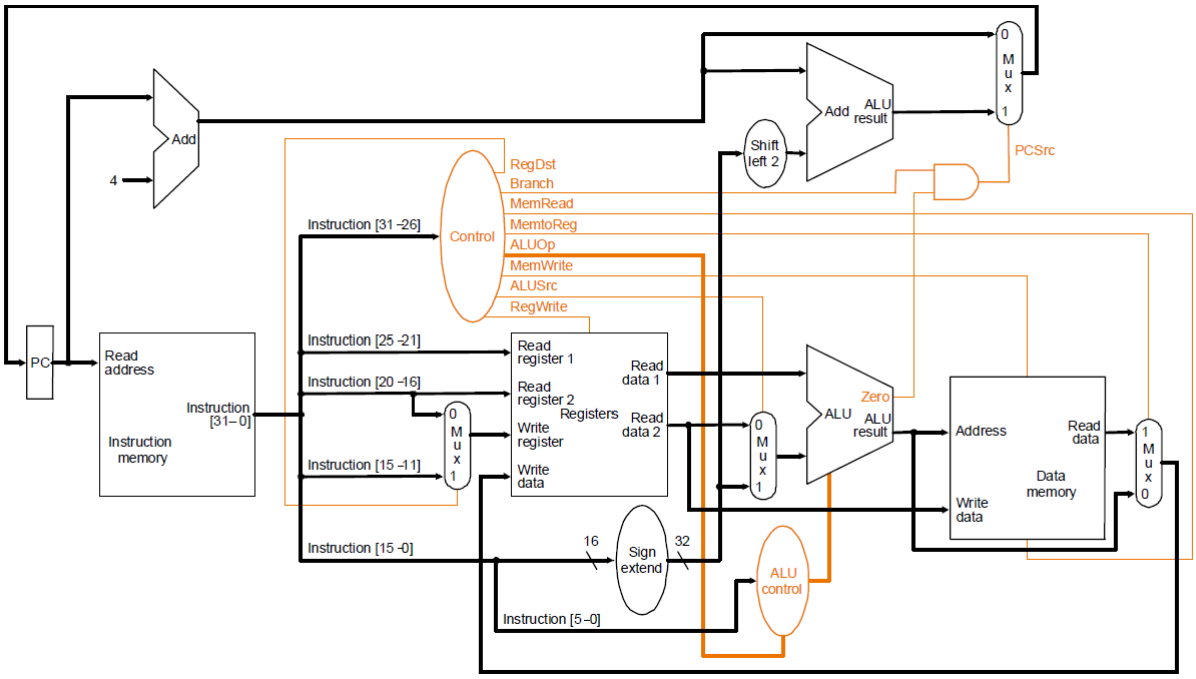



شکل ۸-۲: مفهوم Data Bus

## ۳- معماری پردازنده طراحی شده :

پردازنده‌ی این پروژه در سطح بالا از دو ماژول اصلی تشکیل شده که هر دو در ماژول سطح بالای CPU نمونه سازی می شوند؛ ماژول datapath که تمامی مسیر جریان داده (شامل PC ، حافظه‌ی دستور، بانک رجیستر، sign extend، مالتی‌پلکسرها، ALU، حافظه‌ی داده و منطق محاسبه‌ی آدرس بعدیPC ) را در بر دارد، و ماژول control_unit که بر اساس فیلد opcode دستور جاری (که از خروجی upcode ماژول datapath دریافت می‌شود) ، تمامی سیگنال‌های کنترلی لازم برای هدایت مسیر داده را تولید می‌کند. این دو ماژول از طریق مجموعه‌ای از سیگنال‌های کنترلی دوطرفه به یکدیگر متصل هستند: مسیر داده، فیلد opcode را به واحد کنترل می‌فرستد و واحد کنترل در مقابل، سیگنال‌هایی نظیرALUop ، RegDst، ALUSrc، MemtoReg، RegWrite، MemRead، MemWrite، Branch و Jump را به مسیر داده بازمی‌گرداند تا مالتی‌پلکسرها و واحدهای فعال/غیرفعال‌سازی حافظه به‌درستی هدایت شوند.

همچنین پردازنده طراحی‌شده در این پروژه بر پایه معماری Single Cycle MIPS پیاده‌سازی شده است. در این معماری، هر دستور صرف‌نظر از نوع آن، در یک سیکل کلاک اجرا می‌شود.




شکل ۱-۳: ارتباط مسیر داده (data path) و واحد کنترلی (control unit)

## ۱-۳- مسیر داده (Datapath) :

مسیر داده مهم‌ترین بخش هر پردازنده محسوب می‌شود، زیرا تمامی اطلاعات مورد نیاز برای اجرای دستورها از طریق آن منتقل می‌شوند. در حقیقت Datapath مجموعه‌ای از واحدهای پردازشی، حافظه‌ها، رجیسترها، مالتی‌پلکسرها و گذرگاه‌های انتقال داده است که در کنار یکدیگر امکان اجرای دستورهای مختلف را فراهم می‌کنند.

در پردازنده طراحی‌شده، اجرای هر دستور از Program Counter آغاز می‌شود. مقدار موجود در PC به حافظه دستور ارسال شده و دستور متناظر از حافظه خوانده می‌شود. پس از Fetch دستور، قسمت‌های مختلف آن شامل Opcode ، شماره رجیسترها، مقدار Immediate و فیلد Function استخراج می‌شوند.

در مرحله بعد، واحد Register File بر اساس شماره رجیسترهای موجود در دستور، داده‌های مورد نیاز را در اختیار ALU قرار می‌دهد. اگر دستور از نوع Immediate باشد، مقدار Immediate ابتدا توسط واحد Sign Extend به ۳۲ بیت گسترش یافته و سپس توسط مالتی‌پلکسر به عنوان ورودی دوم ALU انتخاب می‌شود.

واحد ALU با توجه به سیگنال تولیدشده توسط ALU Control عملیات مورد نظر را انجام می‌دهد. نتیجه این عملیات ممکن است مستقیماً در فایل رجیستر ذخیره شود یا برای دسترسی به حافظه داده مورد استفاده قرار گیرد. در دستورهای Load و Store، خروجی ALU به عنوان آدرس حافظه استفاده می‌شود.

در انتهای مسیر، با توجه به نوع دستور، مقدار مناسب از طریق مالتی‌پلکسر Write Back انتخاب شده و در رجیستر مقصد نوشته می‌شود.

همزمان با اجرای دستور، مسیر محاسبه آدرس بعدی نیز فعال است. ابتدا مقدار PC+4 محاسبه شده و سپس در صورت وجود دستورهای Branch یا Jump ، آدرس مقصد تولید و جایگزین می‌شود.

این ساختار موجب می‌شود تمامی مسیرهای مورد نیاز اجرای دستورها به صورت همزمان آماده باشند و تنها با تغییر سیگنال‌های کنترلی، مسیر مناسب انتخاب شود.

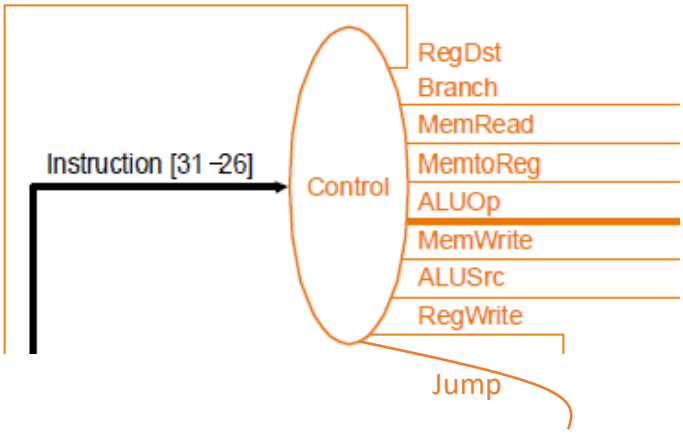
## ۲-۳- واحد کنترلی (Control Unit) :

اگر Datapath را به عنوان بخش اجرایی پردازنده در نظر بگیریم، واحد کنترل نقش مغز تصمیم‌گیرنده سیستم را بر عهده دارد. این واحد با بررسی Opcode موجود در دستور، تشخیص می‌دهد که کدام قسمت‌های مسیر داده باید فعال شوند و هر مالتی‌پلکسر کدام ورودی را انتخاب کند.

در این پروژه، واحد کنترل به صورت یک مدار ترکیبی طراحی شده است؛ بنابراین خروجی‌های آن تنها به Opcode دستور وابسته هستند و نیازی به ذخیره وضعیت داخلی ندارد. این ویژگی باعث کاهش پیچیدگی سخت‌افزار و افزایش سرعت تولید سیگنال‌های کنترلی می‌شود.

از مهم‌ترین سیگنال‌های تولیدشده توسط این واحد می‌توان به RegDst ، ALUSrc، MemRead، MemWrite، MemtoReg، RegWrite، Branch، Jump و ALUOp اشاره کرد. هر یک از این سیگنال‌ها مسئول کنترل یکی از بخش‌های Datapath هستند و ترکیب مناسب آن‌ها باعث اجرای صحیح دستورها می‌شود.

به عنوان مثال، هنگام اجرای دستورLW ، واحد کنترل سیگنال‌های MemRead و MemtoReg را فعال می‌کند تا داده خوانده‌شده از حافظه در فایل رجیستر ذخیره شود. در مقابل، در دستور SW تنها سیگنال MemWrite فعال شده و عملیات نوشتن در حافظه انجام می‌شود.



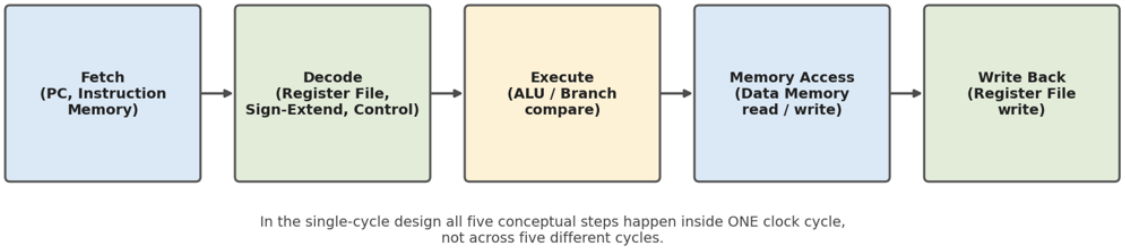

شکل ۲-۳: واحد کنترلی (control unit)

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst | دستور |
|---|---|---|---|---|---|---|---|---|---|
| 0000 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | R-type |
| 0001 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | lw |
| 0010 | 0 | 0 | 1 | 0 | 0 | X | 1 | X | sw |
| 0011 | 0 | 1 | 0 | 0 | 0 | X | 0 | X | beq |
| 0100 | 0 | 1 | 0 | 0 | 0 | X | 0 | X | bnq |
| 0101 | 1 | 0 | 0 | 0 | 0 | X | X | X | jump |
| 0110–1001 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | addi/andi/ori/slti |

جدول ۱-۳: سیگنال های کنترلی برای هر دستور

## ۳-۳- نحوه اجرای دستورات :

اگرچه در پردازنده‌ی تک‌سیکل، تمام مراحل زیر در یک سیکل کلاک واحد و به‌صورت کاملاً ترکیبی رخ می‌دهند، اما از نظر مفهومی می‌توان اجرای هر دستور را به پنج گام تقسیم کرد که در ادامه هر یک به‌طور جداگانه شرح داده شده است:




شکل ۳-۳: پنج گام مفهومی اجرای یک دستور که در پردازنده‌ی تک‌سیکل، همگی درون یک سیکل کلاک انجام می‌شوند

## ۳-۳-۱- مرحله fetch :

در این گام، مقدار جاری PC به‌عنوان آدرس به حافظه‌ی دستور فرستاده می‌شود و دستور ۳۲ بیتی نظیر آن آدرس در خروجی instr قرار می‌گیرد. هم‌زمان، یک جمع‌کننده (ماژول ADD ) مقدار PC+4 را محاسبه می‌کند تا در صورت عدم وقوع پرش، به‌عنوان آدرس دستور بعدی استفاده شود.

## ۳-۳-۲- مرحله Decode :

در گام رمزگشایی، فیلد opcode (بیت‌های ۳۱ تا ۲۶) به واحد کنترل ارسال می‌شود تا سیگنال‌های کنترلی تولید شوند؛ هم‌زمان، فیلدهای rs و rt به بانک رجیستر فرستاده می‌شوند تا مقادیر RF1 و RF2 به‌صورت ترکیبی و بدون تأخیر خوانده شوند، و فیلد ۱۶ بیتی پایینی دستور نیز به‌طور موازی وارد واحد گسترش علامت می‌شود.

## ۳-۳-۳- مرحله Execute :

در گام اجرا، بسته به کلاس دستور، ALU یکی از عملیات جمع، تفریق، AND، OR یا مقایسه‌ی علامت‌دار را روی دو ورودی خود (که یکی همواره RF1 و دیگری بسته به ALUSrc ، RF2 یا خروجی گسترش علامت است) انجام می‌دهد. عملیات دقیق ALU توسط سیگنال سه‌بیتی op که خروجی ماژول ALU_control است، تعیین می‌شود. برای دستورات شاخه (beq/bnq) نیز در همین گام، عملیات تفریق برای تشخیص برابری دو رجیستر (از طریق Zero Flag) انجام می‌شود.

## ۳-۳-۴- مرحله Memory Access :

در گام دسترسی به حافظه که تنها برای دستورات lw و sw معنا دارد، خروجی ALU (که آدرس محاسبه شده است) به حافظه‌ی داده فرستاده می‌شود؛ بسته به سیگنال‌های MemRead/MemWrite ، یا مقداری از حافظه خوانده می‌شود (lw) یا مقدار RF2 در آن آدرس نوشته می‌شود (sw). سایر کلاس‌های دستور (مانند R-type یا branch ) در این گام هیچ فعالیتی روی حافظه‌ی داده انجام نمی‌دهند.

### ۵-۳-۳- مرحله Write Back :

در گام نهایی، اگر سیگنال RegWrite فعال باشد، مقدار نهایی که بسته به سیگنالMemtoReg ، یا خروجی ALU یا خروجی حافظه‌ی داده است در رجیستر مقصدی که توسط مالتی‌پلکسر RegDst انتخاب شده، نوشته می‌شود. این نوشتن، همانند سایر عناصر حالت، تنها روی لبه‌ی مثبت کلاک انجام می‌شود.

## ۴- طراحی و پیاده سازی اجزای پردازنده (ماژول ها) در Verilog :

پس از بررسی ساختار کلی پردازنده و نحوه اجرای دستورها، در این فصل به بررسی جزئیات پیاده‌سازی هر یک از اجزای تشکیل‌دهنده پردازنده پرداخته می‌شود. تمامی بخش‌های پردازنده به صورت ماژول‌های مستقل در زبان توصیف سخت‌افزار Verilog طراحی شده‌اند. این شیوه طراحی علاوه بر افزایش خوانایی و قابلیت توسعه، امکان آزمایش و اشکال‌زدایی هر بخش را به صورت جداگانه فراهم می‌کند.

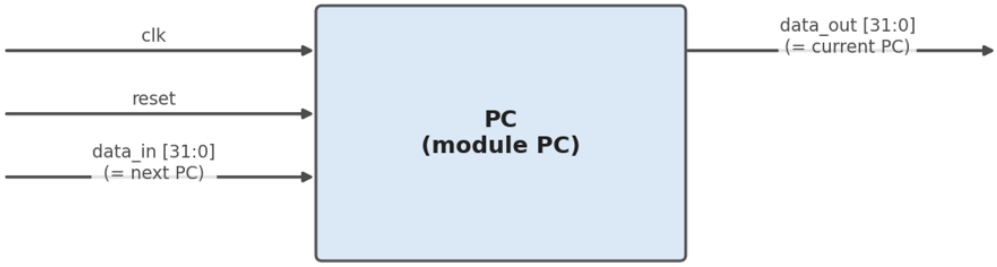
هر ماژول مسئول انجام وظیفه مشخصی در مسیر اجرای دستورها است. برخی از این ماژول‌ها، مانند ALU و مالتی‌پلکسرها، از نوع مدارهای ترکیبی هستند و خروجی آن‌ها تنها به ورودی‌های لحظه‌ای وابسته است؛ در مقابل، ماژول‌هایی مانند Program Counter و Register File دارای عناصر حافظه بوده و عملکرد آن‌ها به سیگنال کلاک وابسته است.

در ادامه، هر یک از ماژول‌های استفاده‌شده در طراحی پردازنده معرفی شده و ساختار داخلی، عملکرد، ارتباط با سایر بخش‌های پردازنده و نحوه پیاده‌سازی آن‌ها بررسی می‌شود.

## ۱-۴- ماژول Program Counter (PC) :

Program Counter یا PC نقطه آغاز مسیر داده محسوب می‌شود که وظیفه نگهداری آدرس دستور جاری را بر عهده دارد. اجرای هر برنامه از مقدار اولیه موجود در این رجیستر آغاز می‌شود و در هر سیکل کلاک، مقدار آن به گونه‌ای تغییر می‌کند که آدرس دستور بعدی مشخص شود و مقدار خروجی آن مستقیماً به Instruction Memory ارسال شده و باعث انتخاب دستور بعدی می‌شود.

در معماری MIPS ، مقدار PC معمولاً چهار واحد افزایش می‌یابد، چرا که هر دستور در ۴ خانه ذخیره می شود. البته در صورت اجرای دستورهای Branch یا Jump، مقدار جدید PC توسط مجموعه‌ای از جمع‌کننده‌ها، مالتی‌پلکسرها و منطق مربوط به دستورهای Branch و Jump محاسبه شده و جایگزین مقدار عادی PC+4 می‌شود.




شکل ۱-۴: ماژول PC

```
module PC(
    input clk,
    input reset,
    input [31:0] data_in,
    output reg [31:0] data_out
    );

   always @(posedge clk or posedge reset)
   begin
   if (reset==1)
      data_out = 0;
   else
      data_out = data_in;
   end

endmodule
```

کد ۱-۴: ماژول PC

در پیاده‌سازی انجام‌شده، از یک بلوک always حساس به لبه بالارونده کلاک و همچنین سیگنال Reset استفاده شده است. این ساختار باعث می‌شود مقدار PC تنها در زمان مناسب تغییر کند و از ایجاد وضعیت‌های ناپایدار جلوگیری شود.

در ابتدای اجرای برنامه، اگر سیگنال Reset فعال باشد، مقدار خروجی برابر صفر قرار داده می‌شود. این عمل باعث می‌شود اجرای برنامه همواره از اولین خانه حافظه دستور آغاز شود. پس از غیرفعال شدن Reset ، در هر لبه بالارونده کلاک، مقدار موجود در ورودی data_in در رجیستر PC ذخیره شده و در خروجی قرار می‌گیرد.

این پیاده‌سازی کاملاً با رفتار مورد انتظار یک رجیستر ترتیبی در معماری MIPS مطابقت دارد و به دلیل استفاده از کلاک، از تغییرات ناخواسته مقدار PC در طول سیکل جلوگیری می‌کند.

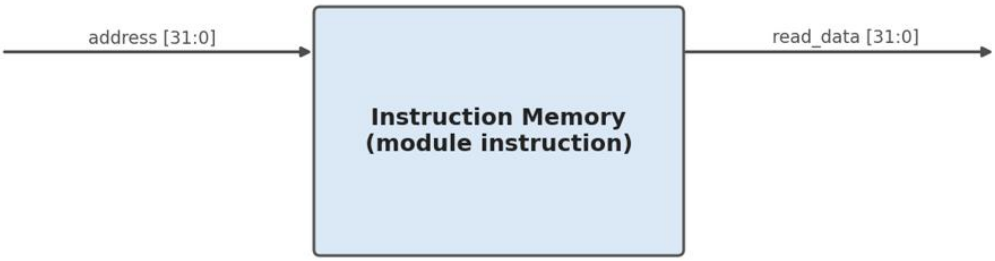
ورودی PC از خروجی آخرین مالتی‌پلکسر مسیر داده تأمین می‌شود که وظیفه انتخاب میان آدرس PC+4 ، آدرس Branch و آدرس Jump را بر عهده دارد. خروجی PC نیز مستقیماً به حافظه دستور متصل شده و آدرس دستور جاری را مشخص می‌کند. بنابراین، این ماژول در آغاز و پایان چرخه اجرای هر دستور حضور دارد و نقش کلیدی در کنترل جریان اجرای برنامه ایفا می‌کند.

## ۲-۴- ماژول Instruction Memory :

حافظه دستور (Instruction Memory) یکی از مهم‌ترین اجزای مسیر داده در پردازنده‌های مبتنی بر معماری MIPS است. این واحد وظیفه نگهداری تمامی دستورهای برنامه و ارسال دستور مناسب به پردازنده در هر سیکل کلاک را بر عهده دارد. در واقع، فرآیند اجرای هر دستور با مراجعه پردازنده به حافظه دستور آغاز می‌شود؛ بنابراین این واحد نقطه شروع چرخه اجرای دستورها محسوب می‌شود.

در معماری تک‌سیکل، در ابتدای هر سیکل، مقدار موجود در Program Counter (PC) به عنوان آدرس حافظه دستور مورد استفاده قرار می‌گیرد. حافظه دستور با استفاده از این آدرس، دستور متناظر را بازیابی کرده و خروجی حافظه مستقیماً به واحد Register File، واحد Control Unit و واحد Sign Extend متصل می‌شود تا هر بخش، فیلد مورد نیاز خود را از دستور استخراج کند.

در معماری MIPS ، تمام دستورها دارای طول ثابت **۳۲ بیت** هستند. این ویژگی باعث می‌شود که پردازنده بتواند به سادگی آدرس دستور بعدی را با افزایش چهار واحدی مقدار PC محاسبه کند. همچنین در این پروژه، حافظه دستور به صورت آرایه‌ای از ۱۲۸ کلمه ۳۲ بیتی تعریف شده است. هر خانه حافظه قادر به نگهداری یک دستور کامل MIPS است. بنابراین ظرفیت کل حافظه برابر با ۱۲۸ دستور خواهد بود.




شکل ۲-۴: ماژول Instruction Memory

```
module instruction(
    input [31:0] address,
    output [31:0] read_data
    );

   reg [31:0] mem_array [127:0];

   assign read_data = mem_array[address[31:2]];

   initial
   begin
      $readmemb ("instrmem_test.txt",mem_array);
   end
endmodule
```

کد ۲-۴: ماژول Instruction Memory

در این ماژول، حافظه دستور به صورت آرایه‌ای از ۱۲۸ کلمه ۳۲ بیتی تعریف شده است که وظیفه نگهداری دستورهای برنامه را بر عهده دارد. دستورات در ابتدای شبیه‌سازی با استفاده از تابع $readmemb از فایل متنی که برای تست نوشته شده بارگذاری می‌شوند. همچنین، عملیات خواندن حافظه به صورت ترکیبی انجام می‌شود؛ بنابراین با تغییر آدرس ورودی، دستور متناظر بدون نیاز به سیگنال کلاک در خروجی قرار می‌گیرد. برای انتخاب خانه حافظه از بیت‌های address[31:2] استفاده شده است که با توجه به هم‌تراز بودن دستورهای ۳۲ بیتی، روشی استاندارد برای تبدیل آدرس به صورت بایتی به آدرس به صورت word در معماری MIPS محسوب می‌شود.

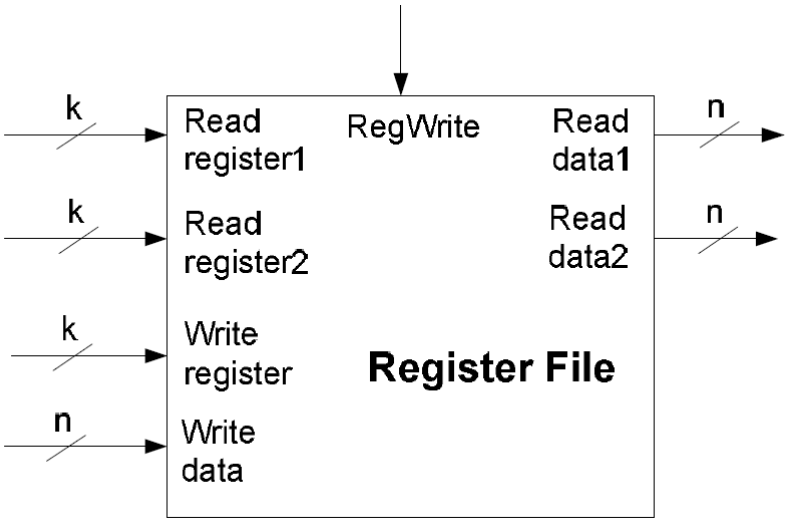
خروجی این ماژول مستقیماً در اختیار واحد کنترل، فایل رجیستر، واحد توسعه علامت و واحد کنترل ALU قرار می‌گیرد و در واقع آغازگر چرخه اجرای هر دستور در پردازنده است.

## ۳-۴- ماژول Register File :

در معماری MIPS ، Register File شامل ۳۲ رجیستر عمومی ۳۲ بیتی است. هر رجیستر دارای یک شماره اختصاصی از صفر تا سی و یک است که از طریق آدرس‌های پنج بیتی قابل انتخاب خواهد بود و هر یک قادر به ذخیره یک word اطلاعات هستند. وجود این رجیسترها موجب می‌شود پردازنده بتواند به جای مراجعه مکرر به حافظه اصلی، داده‌های موردنیاز را با سرعت بسیار بیشتری در اختیار واحد محاسباتی قرار دهد. این موضوع یکی از عوامل اصلی افزایش کارایی پردازنده است.

در این پروژه، فایل ثبات دارای دو درگاه خواندن و یک درگاه نوشتن است. به همین دلیل، پردازنده قادر است به طور هم‌زمان دو داده مستقل را از دو رجیستر مختلف دریافت کرده و در صورت نیاز، نتیجه عملیات را در رجیستر دیگری ذخیره کند.

وجود دو درگاه خواندن برای اجرای دستورهای نوع R و بسیاری از دستورهای دیگر ضروری است؛ زیرا اغلب این دستورها به دو عملوند نیاز دارند. همچنین وجود تنها یک درگاه نوشتن با ساختار مجموعه دستورهای MIPS سازگار است، زیرا هر دستور در نهایت تنها یک نتیجه تولید می‌کند.




شکل ۳-۴: ماژول Register File

| نام سیگنال | جهت | بیت | توضیح |
|---|---|---|---|
| clk | ورودی | ۱ | سیگنال کلاک |
| read_addr1 / read_addr2 | ورودی | ۵ | آدرس رجیستر اول و دوم |
| write_addr | ورودی | ۵ | آدرس رجیستر مقصد |
| write_data | ورودی | ۳۲ | داده ورودی برای نوشتن |
| regwrite | ورودی | ۱ | فعال کننده عملیات نوشتن |
| read_data1 / read_data2 | خروجی | ۳۲ | داده خوانده شده از رجیستر اول و دوم |

جدول ۱-۴: مشخصات سیگنال های ماژول Register File

```
module Register_file(
    input clk,
    input [4:0] read_addr1,
    input [4:0] read_addr2,
    input [4:0] write_addr,
    input [31:0] write_data,
    output [31:0] read_data1,
    output [31:0] read_data2,
    input regwrite
    );

    reg [31:0] regfile [31:0];
    initial regfile[0] = 0;

    always @(posedge clk)
    begin
      regfile[0] = 0;
      if(regwrite)
         regfile[write_addr] <= write_data;
    end
     assign read_data1 = regfile[read_addr1];
     assign read_data2 = regfile[read_addr2];


endmodule
```

کد ۳-۴: ماژول Register File

در این ماژول، فایل ثبات به صورت آرایه‌ای شامل ۳۲ رجیستر ۳۲ بیتی تعریف شده است که هر عنصر آن متناظر با یکی از رجیسترهای عمومی پردازنده است. عملیات خواندن رجیسترها به صورت ترکیبی انجام می‌شود؛ به این معنا که با تغییر آدرس‌های ورودی، داده متناظر بدون انتظار برای لبه کلاک در خروجی ظاهر می‌شود. این ویژگی موجب می‌شود عملوندهای موردنیاز ALU در همان سیکل اجرای دستور در دسترس قرار گیرند.

در مقابل، عملیات نوشتن به صورت ترتیبی و تنها در لبه بالارونده سیگنال کلاک انجام می‌شود. همچنین، نوشتن اطلاعات تنها زمانی صورت می‌گیرد که سیگنال RegWrite فعال باشد. این روش از تغییر ناخواسته محتویات رجیسترها جلوگیری کرده و عملکرد فایل ثبات را با معماری تک‌سیکل MIPS هماهنگ می‌سازد.

یکی از ویژگی‌های مهم این پیاده‌سازی، ثابت نگه داشتن مقدار رجیستر شماره صفر است. در معماریMIPS ، رجیستر $zero همواره مقدار صفر را نگهداری می‌کند و امکان تغییر آن وجود ندارد. به همین دلیل، در ابتدای هر سیکل کلاک مقدار این رجیستر مجدداً برابر صفر قرار داده شده است تا حتی در صورت تلاش برای نوشتن روی آن، مقدارش تغییر نکند.

خروجی‌های این ماژول مستقیماً به واحد ALU متصل می‌شوند و عملوندهای موردنیاز عملیات محاسباتی را فراهم می‌کنند. همچنین داده تولیدشده توسط ALU یا حافظه داده، از طریق مسیر Write Back دوباره به ورودی فایل ثبات بازمی‌گردد تا در رجیستر مقصد ذخیره شود. بنابراین فایل ثبات هم در ابتدای چرخه اجرای دستور (خواندن عملوندها) و هم در انتهای آن (نوشتن نتیجه) نقش اساسی ایفا می‌کند.

## ۴-۴- ماژول Sign Extend :

در معماری MIPS ، بسیاری از دستورها از قالب I-Type استفاده می‌کنند. در این نوع دستورها، یکی از عملوندها به جای آنکه از فایل ثبات خوانده شود، به صورت یک مقدار ثابت ۱۶ بیتی (Immediate) در داخل خود دستور قرار دارد. از آنجا که واحد ALU در این پروژه بر روی داده‌های ۳۲ بیتی عمل می‌کند، لازم است این مقدار ۱۶ بیتی پیش از ورود به ALU به یک مقدار ۳۲ بیتی تبدیل شود.

وظیفه انجام این تبدیل بر عهده واحد Sign Extend است. این واحد مقدار Immediate را بررسی کرده و با توجه به بیت علامت، آن را به یک عدد ۳۲ بیتی معادل تبدیل می‌کند. بدین ترتیب، مقادیر مثبت و منفی بدون تغییر مفهوم عددی در عملیات بعدی مورد استفاده قرار می‌گیرند.

در این پروژه، واحد Sign Extend برای اجرای دستورهای ADDI، LW، SW، BEQ، BNQ و SLTI مورد استفاده قرار می‌گیرد.

شکل ۴-۴: ماژول Sign extend

```
module sign_extend(
    input [15:0] data_in,
    output reg[31:0] data_out
    );

   always @(*)
   begin
      if (~data_in[15])
         data_out = {16'b0,data_in};
      else
         data_out = {16'b1111111111111111,data_in};
   end
endmodule
```

کد ۴-۴: ماژول Sign extend

در این ماژول، بیت شانزدهم (بیت علامت) مقدار Immediate بررسی می‌شود. اگر این بیت صفر باشد، مقدار ورودی یک عدد مثبت در نظر گرفته شده و ۱۶ بیت بالایی خروجی با صفر پر می‌شود. در مقابل، اگر بیت علامت برابر یک باشد، مقدار ورودی یک عدد منفی محسوب شده و ۱۶ بیت بالایی خروجی با یک مقداردهی می‌شود تا نمایش عدد به صورت مکمل دو حفظ شود.

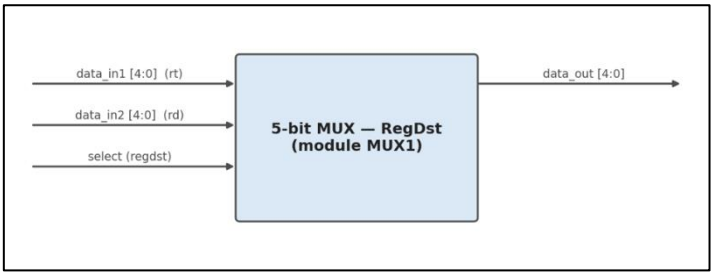
خروجی این واحد مستقیماً به یکی از ورودی‌های مالتی‌پلکسر انتخاب عملوند ALU متصل است. در دستورهای Immediate، Load، Store و Branch، با فعال شدن سیگنال ALUSrc، این مقدار به عنوان ورودی دوم ALU انتخاب می‌شود. در نتیجه، واحد Sign Extend نقش مهمی در آماده‌سازی داده‌های موردنیاز برای انجام عملیات محاسباتی و محاسبه آدرس حافظه ایفا می‌کند.

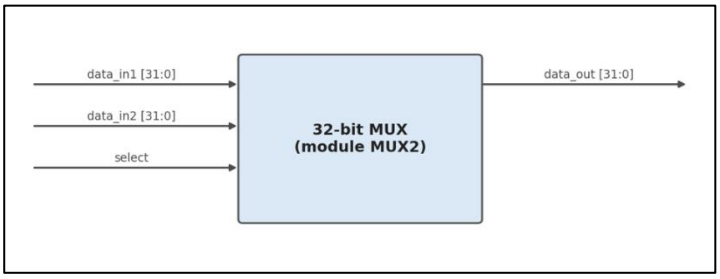
## ۵-۴- ماژول MUX :

در طول اجرای دستورهای مختلف، مسیر حرکت داده در پردازنده همواره یکسان نیست. برای مثال، گاهی ورودی دوم ALU از فایل ثبات تأمین می‌شود و گاهی از مقدار Immediate همچنین در برخی دستورها نتیجه نهایی از خروجی ALU و در برخی دیگر از حافظه داده دریافت می‌شود. به منظور انتخاب مسیر مناسب در هر لحظه، از مالتی‌پلکسرها (Multiplexer) استفاده می‌شود.

مالتی‌پلکسر یک مدار ترکیبی است که بر اساس مقدار سیگنال انتخاب (Select) ، یکی از ورودی‌ها را به خروجی منتقل می‌کند. در پردازنده طراحی‌شده، چندین مالتی‌پلکسر در بخش‌های مختلف Datapath به کار رفته‌اند و تمامی آن‌ها توسط سیگنال‌های تولیدشده در واحد کنترل هدایت می‌شوند.

در این پروژه دو نوع مالتی‌پلکسر طراحی شده است: ۱) MUX1 برای انتخاب رجیستر مقصد با ۵ بیت داده و ۲) MUX2 برای انتخاب رجیستر مقصد با ۳۲ بیت داده. هر دو ماژول از نظر عملکرد مشابه هستند و تنها در عرض داده تفاوت دارند.




```
module MUX1(
    input [4:0] data_in1,
    input [4:0] data_in2,
    input select,
    output reg[4:0] data_out
    );

   always @(*)
   begin
      if(~select)
         data_out = data_in1;
      else
         data_out = data_in2;

   end
endmodule
```

شکل و کد ۵-۴: ماژول MUX1

```
module MUX2(
    input [31:0] data_in1,
    input [31:0] data_in2,
    input select,
    output reg[31:0] data_out
    );

   always @(*)
   begin
      if(~select)
         data_out = data_in1;
      else
         data_out = data_in2;

   end
endmodule
```

شکل و کد ۶-۴: ماژول MUX2

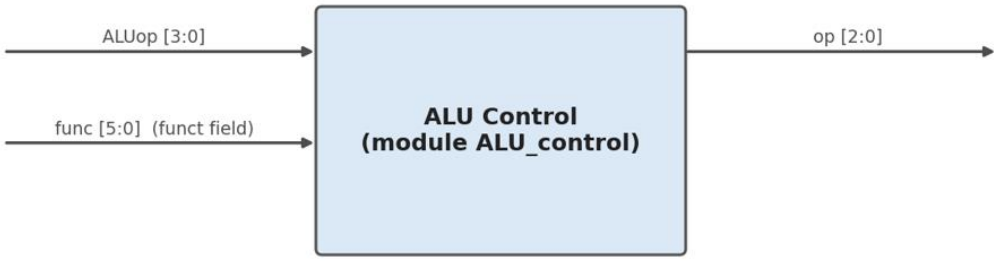
هر دو مالتی‌پلکسر به صورت مدارهای ترکیبی پیاده‌سازی شده‌اند و خروجی آن‌ها تنها به مقدار ورودی‌ها و سیگنال انتخاب وابسته است. در صورتی که سیگنال انتخاب برابر صفر باشد، ورودی اول به خروجی منتقل می‌شود و در غیر این صورت ورودی دوم انتخاب خواهد شد.

در پردازنده طراحی‌شده، از این مالتی‌پلکسرها برای انتخاب رجیستر مقصد، تعیین ورودی دوم ALU، انتخاب داده مرحله Write Back، انتخاب آدرس Branch و انتخاب آدرس Jump استفاده شده است. به همین دلیل، این واحدها نقش مهمی در تغییر مسیر حرکت داده متناسب با نوع دستور ایفا می‌کنند و بخش عمده انعطاف‌پذیری Datapath را فراهم می‌سازند.

## ۶-۴- ماژول ALU Control :

واحد ALU Control یکی از مهم‌ترین بخش‌های مسیر داده در پردازنده است که وظیفه تعیین نوع عملیات قابل اجرا توسط واحد حسابی و منطقی (ALU) را بر عهده دارد. اگرچه واحد کنترل اصلی (Control Unit) نوع کلی دستور را از روی Opcode تشخیص می‌دهد، اما در بسیاری از دستورها، به‌ویژه دستورهای نوع R-Type، تعیین عملیات دقیق ALU تنها از طریق Opcode امکان‌پذیر نیست. به همین دلیل، واحدی مجزا به نام ALU Control برای تفسیر دقیق‌تر دستورها در نظر گرفته می‌شود.

در معماری MIPS، برای دستورهای R-Type، تمامی دستورها دارای Opcode یکسان (000000) هستند و نوع عملیات از طریق فیلد Function مشخص می‌شود. بنابراین ALU Control با بررسی هم‌زمان سیگنال ALUOp تولیدشده توسط واحد کنترل و فیلد Function موجود در دستور، کد کنترلی مناسب برای ALU را تولید می‌کند.




شکل ۷-۴: ماژول ALU Control

```
module ALU_control(
    input [3:0] ALUop,
    input [5:0] func,
    output reg[2:0] op
    );

   always @(*)
   begin
      casex({ALUop,func})
      10'b0000100000 : op = 3'b000; // R type add
      10'b0000100010 : op = 3'b001; // R type sub
      10'b0000100100 : op = 3'b010; // R type and
      10'b0000100101 : op = 3'b011; // R type or
      10'b0000101010 : op = 3'b100; // R type compare
      10'b0001xxxxxx : op = 3'b000; // LW
      10'b0010xxxxxx : op = 3'b000; // SW
      10'b0011xxxxxx : op = 3'b001; // beq
      10'b0100xxxxxx : op = 3'b001; // bnq
      10'b0101xxxxxx : op = 3'b000; // jump
      10'b0110xxxxxx : op = 3'b000; // add immediate
      10'b0111xxxxxx : op = 3'b010; // and immediate
      10'b1000xxxxxx : op = 3'b011; // or immediate
      10'b1001xxxxxx : op = 3'b100; // compare immediate


      endcase
   end

endmodule
```

کد ۷-۴: ماژول ALU Control

در این پروژه، واحد ALU Control به صورت یک مدار ترکیبی پیاده‌سازی شده است. تصمیم‌گیری این واحد بر اساس ترکیب دو ورودی ALUOp و Function انجام می‌شود. برای این منظور از دستور casex استفاده شده است که امکان استفاده از بیت‌های بی‌اهمیت (Don't Care) را نیز فراهم می‌کند.

برای دستورهای نوع R-Type، مقدار ALUOp تنها نشان‌دهنده این است که نوع دستور از خانواده R است و تعیین عملیات نهایی از طریق فیلد Function انجام می‌شود. بر این اساس، عملیاتی مانند جمع، تفریق، AND، OR و مقایسه به کدهای کنترلی متناظر تبدیل شده و به ALU ارسال می‌شوند.

در مقابل، برای دستورهای LW، SW، BEQ، BNQ، JUMP و Immediate ، نوع عملیات مستقیماً از مقدار ALUOp مشخص می‌شود و دیگر نیازی به بررسی فیلد Function وجود ندارد. این موضوع باعث ساده‌تر شدن منطق تصمیم‌گیری و کاهش پیچیدگی مدار شده است.

خروجی این واحد مستقیماً به ALU متصل است و تعیین می‌کند که واحد حسابی و منطقی چه عملیاتی را بر روی ورودی‌های خود انجام دهد. به همین دلیل، ALU Control به عنوان واسط میان واحد کنترل اصلی و ALU عمل می‌کند و نقش مهمی در هماهنگی عملکرد این دو بخش دارد.

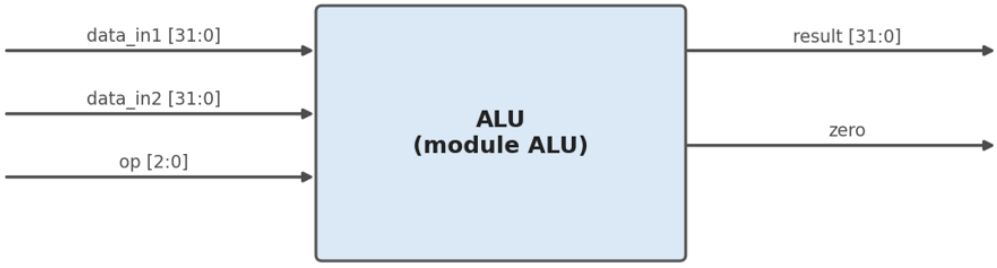
| نوع عملیات | op | function | نوع دستور | ALUop |
|---|---|---|---|---|
| add | 000 | 100000 | R-type | 0000 |
| subtract | 001 | 100010 | R-type | 0000 |
| and | 010 | 100100 | R-type | 0000 |
| or | 011 | 100101 | R-type | 0000 |
| slt (مقایسه) | 100 | 101010 | R-type | 0000 |
| add (محاسبه آدرس) | 000 | xxxxxx | lw | 0001 |
| add (محاسبه آدرس) | 000 | xxxxxx | sw | 0010 |
| subtract (مقایسه تساوی) | 001 | xxxxxx | beq | 0011 |
| subtract (مقایسه تساوی) | 001 | xxxxxx | bnq | 0100 |
| - | 000 | xxxxxx | jump | 0101 |
| Add immediate | 000 | xxxxxx | addi | 0110 |
| And immediate | 010 | xxxxxx | andi | 0111 |
| Or immediate | 011 | xxxxxx | ori | 1000 |
| slt immediate | 100 | xxxxxx | slti | 1001 |

جدول ۲-۴: نگاشت ALUOp و Function به عملیات ALU

## ۷-۴- ماژول ALU :

واحد حسابی و منطقی (Arithmetic Logic Unit یا ALU) مهم‌ترین بخش پردازشی هر پردازنده محسوب می‌شود. تمامی عملیات ریاضی، منطقی و مقایسه‌ای مورد نیاز دستورها در این واحد انجام می‌شود و نتیجه آن برای ادامه فرآیند اجرای دستور در اختیار سایر بخش‌های پردازنده قرار می‌گیرد.

در پردازنده طراحی‌شده، ALU قابلیت اجرای عملیات جمع، تفریق، AND، OR و مقایسه را دارد. همچنین این واحد سیگنال **Zero** را نیز تولید می‌کند که در اجرای دستورهای انشعاب مانند **BEQ** و **BNQ** مورد استفاده قرار می‌گیرد.




شکل ۸-۴: ماژول ALU

```
module ALU(
    input [31:0] data_in1,
    input [31:0] data_in2,
    input [2:0] op,
    output reg[31:0] result,
    output reg zero
    );
    reg data_compare;

    always @(*)
    begin
      case(op)
      3'b000 : begin result = data_in1 + data_in2; end
      3'b001 : begin result = data_in1 - data_in2; end
      3'b010 : begin result = data_in1 & data_in2; end
      3'b011 : begin result = data_in1 | data_in2; end
      3'b100 : begin if(data_compare == 1) result = 1; else result = 0; end
      default : result = 0;
      endcase
    end

   always @(*)
   begin
      case({data_in1[31],data_in2[31]})
      2'b00 : begin if (data_in1 < data_in2) data_compare=1; else data_compare=0; end
      2'b01 : data_compare=0;
      2'b10 : data_compare=1;
      2'b11 : begin if (~data_in1 + 1 > ~data_in2 + 1) data_compare=1; else data_compare=0; end
      endcase
   end

   always @(*)
   begin
      if(result==0)
         zero = 1;
      else
         zero = 0;
   end

endmodule
```

کد ۸-۴: ماژول ALU

در این پروژه، ALU به صورت یک مدار ترکیبی طراحی شده است و خروجی آن تنها به مقادیر ورودی و کد عملیات وابسته است. عملیات قابل انجام شامل جمع، تفریق، AND، OR و مقایسه بوده که هر یک بر اساس کد دریافتی (op) از واحد ALU Control انتخاب می‌شوند.

در پیاده سازی این ماژول، عمل مقایسه از سایر عملیات ها جدا شده است. برای اجرای دستورهای SLT و SLTI، ابتدا وضعیت دو عملوند با در نظر گرفتن علامت آن‌ها بررسی می‌شود (case({data_in1[31], data_in2[31]})) . اگر یکی مثبت و دیگری منفی باشد، نتیجه بدون نیاز به هیچ محاسبه‌ای مشخص است (عدد منفی همواره کوچک‌تر است)؛ تنها زمانی‌که هر دو عملوند هم‌علامت باشند (هر دو مثبت یا هر دو منفی)، مقایسه‌ی مقدار مطلق (~data_in1+1 در برابر ~data_in2+1 برای حالت هر دو منفی) انجام می‌شود. این روش باعث می‌شود عملیات مقایسه اعداد علامت‌دار با دقت بیشتری انجام شود.

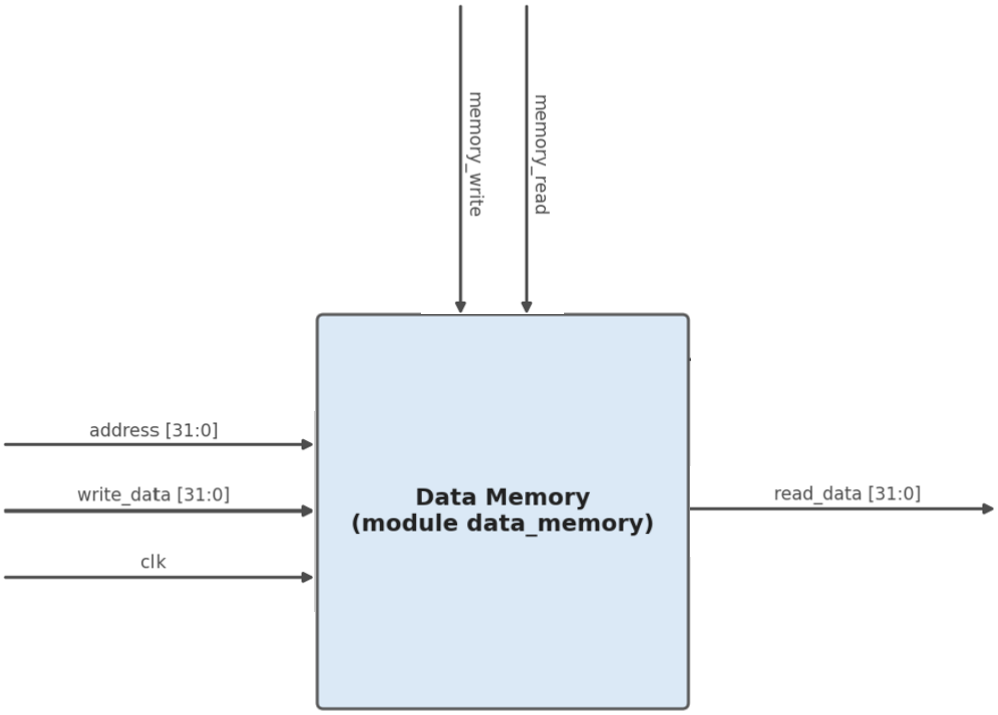
پس از انجام عملیات اصلی، مقدار خروجی بررسی شده و در صورتی که نتیجه برابر صفر باشد، سیگنال Zero فعال می‌شود. این سیگنال در ادامه مسیر داده برای تصمیم‌گیری دستورهای Branch مورد استفاده قرار می‌گیرد. در پروژه حاضر، منطق انتخاب بین دستورهای BEQ و BNQ در Datapath و با استفاده از سیگنال Zero پیاده‌سازی شده است؛ بنابراین ALU برای تعیین انتخاب میان این دو دستور تنها وظیفه تولید نتیجه عملیات و وضعیت صفر بودن آن را بر عهده دارد.

خروجی این ماژول بسته به نوع دستور، یا مستقیماً در فایل ثبات ذخیره می‌شود و یا به عنوان آدرس حافظه داده برای دستورهای Load و Store مورد استفاده قرار می‌گیرد. به همین دلیل، ALU در مرکز مسیر داده قرار گرفته و تقریباً تمامی دستورهای پردازنده به نوعی از این واحد استفاده می‌کنند.

## ۸-۴- ماژول Data Memory :

برخلاف Instruction Memory که تنها برای نگهداری برنامه استفاده می‌شود، Data Memory در طول اجرای برنامه به صورت پویا مورد خواندن و نوشتن قرار می‌گیرد. در معماری MIPS ، دستور LW (Load Word) برای خواندن داده از حافظه و دستور SW (Store Word) برای ذخیره داده در حافظه استفاده می‌شود. هر دو دستور ابتدا Effective Address را با استفاده از واحد ALU محاسبه کرده و سپس این آدرس به Data Memory ارسال می‌شود. بر این اساس، حافظه داده یکی از مهم‌ترین بخش‌های مرحله دسترسی به حافظه (Memory Access) محسوب می‌شود.

در پروژه حاضر، حافظه داده به صورت آرایه‌ای از ۱۲۸ کلمه ۳۲ بیتی پیاده‌سازی شده و امکان انجام عملیات خواندن و نوشتن را بر اساس سیگنال‌های کنترلی فراهم می‌کند.




شکل ۹-۴: ماژول Data Memory

```
module data_memory(
    input [31:0] address,
    input [31:0] write_data,
    input clk,
    input memory_read,
    input memory_write,
    output reg [31:0] read_data
    );
   reg [31:0] mem_array [127:0];
   initial mem_array[101] = 34;

   always @(*)
   begin
      if(memory_read && ~memory_write)
         read_data = mem_array[address];
   end

   always @(posedge clk)
   begin
      if(~memory_read && memory_write)
         mem_array[address] = write_data;

   end

endmodule
```

کد ۹-۴: ماژول Data Memory

در این ماژول، حافظه داده به صورت آرایه‌ای از ۱۲۸ word ۳۲ بیتی تعریف شده است که امکان ذخیره داده‌های موردنیاز برنامه را فراهم می‌کند. عملیات خواندن و نوشتن با استفاده از دو سیگنال کنترلی MemoryRead و MemoryWrite کنترل می‌شود تا در هر لحظه تنها یکی از این دو عملیات انجام گیرد.

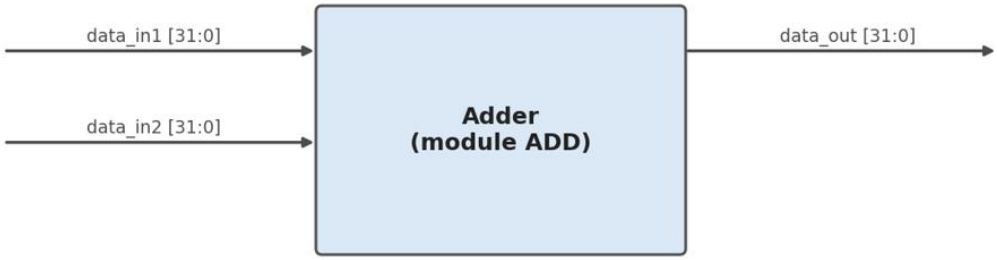
خواندن داده از حافظه به صورت ترکیبی (Combinational) پیاده‌سازی شده است؛ به این معنا که با فعال شدن سیگنال memory_read و فعال نبودن memory_write و ارائه آدرس معتبر، محتوای آدرس بدون انتظار برای لبه کلاک در خروجی قرار می‌گیرد. در مقابل، عملیات نوشتن **ترتیبی** (Sequential) بوده و تنها در لبه بالارونده سیگنال کلاک انجام می‌شود. همچنین مقداردهی اولیه‌ی mem_array[101] = 34 نیز برای فراهم کردن یک مقدار قابل مشاهده در شبیه‌سازی درج شده است.

آدرس ورودی این ماژول از خروجی ALU تأمین می‌شود؛ زیرا ALU پیش از دسترسی به حافظه، Effective Address را محاسبه می‌کند. همچنین داده‌ای که در دستور SW در حافظه ذخیره می‌شود، از خروجی دوم فایل ثبات دریافت می‌گردد. در مقابل، داده خوانده‌شده در دستور LW از طریق مالتی‌پلکسر مرحله Write Back به Register File بازگردانده می‌شود. بنابراین این ماژول حلقه ارتباطی میانALU ، Register File و حافظه را تشکیل داده و اجرای صحیح دستورهای Load و Store را امکان‌پذیر می‌سازد.

## ۹-۴- ماژول Adder :

در این پروژه، عملیات جمع تنها به واحد ALU محدود نمی‌شود. در بخش‌های مختلف مسیر داده نیز برای محاسبه آدرس دستور بعدی و همچنین تعیین مقصد دستورهای Branch از واحدهای جمع‌کننده مستقل استفاده می‌شود. به همین دلیل، در طراحی حاضر یک ماژول مجزا برای انجام عملیات جمع در نظر گرفته شده است.

این ماژول در دو نقطه از مسیر داده مورد استفاده قرار می‌گیرد. نخست برای محاسبه مقدار PC + 4 که آدرس دستور بعدی را تعیین می‌کند و دوم برای محاسبه آدرس مقصد دستورهای Branch که از جمع مقدار PC + 4 و آفست شیفت‌داده‌شده به دست می‌آید.




شکل ۱۰-۴: ماژول Adder

```
module ADD(
    input [31:0] data_in1,
    input [31:0] data_in2,
    output [31:0] data_out
    );

   assign data_out = data_in1 + data_in2;

endmodule
```

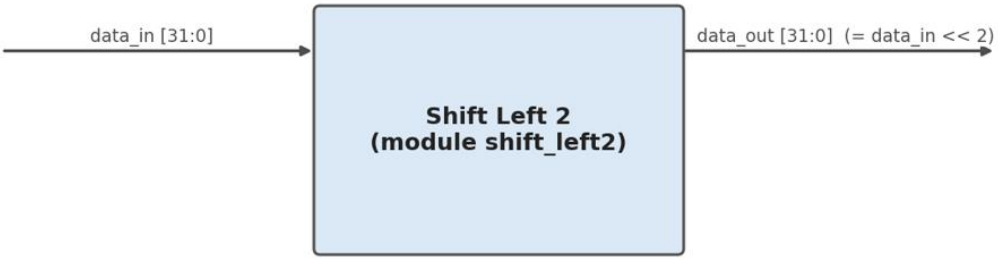
کد ۱۰-۴: ماژول Adder

این ماژول ساده‌ترین بخش طراحی پردازنده است و تنها وظیفه جمع دو مقدار ۳۲ بیتی را بر عهده دارد. پیاده‌سازی آن به صورت یک مدار ترکیبی انجام شده و خروجی همواره برابر حاصل جمع دو ورودی است.

در پروژه حاضر، دو نمونه از این ماژول در Datapath استفاده شده است. نمونه اول برای محاسبه مقدار PC + 4 و نمونه دوم برای تعیین آدرس مقصد دستورهای انشعاب به کار رفته است. استفاده از ماژولی مستقل برای عملیات جمع، علاوه بر افزایش خوانایی طراحی، امکان استفاده مجدد از آن را در بخش‌های مختلف پردازنده فراهم می کند.

## ۱۰-۴- ماژول Shift Left 2 :

در معماری این پروژه، برخی از آدرس‌ها پیش از استفاده باید دو بیت به سمت چپ انتقال داده شوند. این عملیات در دو بخش از پردازنده مورد استفاده قرار می‌گیرد. نخست هنگام محاسبه مقصد دستورهای Branch و دوم در تشکیل آدرس مقصد دستور Jump.




شکل ۱۱-۴: ماژول Shift Left 2

```
module shift_left2(
    input [31:0] data_in,
    output [31:0] data_out
    );

   assign data_out = {data_in[29:0],2'b0};
endmodule
```

کد ۱۱-۴: ماژول Shift Left 2

عملکرد این ماژول بر پایه یک انتقال منطقی دو بیتی به سمت چپ است و هیچ‌گونه عملیات محاسباتی پیچیده‌ای در آن انجام نمی‌شود و عملا مانند ضرب ورودی در ۴ است.

در طراحی حاضر، از این ماژول در دو بخش Datapath استفاده شده است. در مسیر اجرای دستورهای Branch، مقدار Immediate توسعه‌یافته ابتدا وارد این واحد شده و سپس به جمع‌کننده مقصد انشعاب ارسال می‌شود. همچنین در مسیر اجرای دستور Jump نیز فیلد آدرس دستور پیش از ترکیب با چهار بیت پرارزش PC از این واحد عبور می‌کند.

## ۱۱-۴- ماژول Datapath :

عملکرد این ماژول در فصل ۳ توضیح داده شد و از آنجا که مسیر حرکت داده برای هر گروه از دستورها متفاوت است، تحلیل دقیق نحوه اجرای دستورهای R-Type ، Immediate، Load/Store، Branch و Jump در فصل پنجم و در قالب بررسی مسیر اجرای دستورها ارائه خواهد شد. در اینجا تنها کد آن را بررسی می کنیم:

```
module datapath(
    input clk,
    input reset,
    input jump,
    input branch,
    input memory_read,
    input memory_write,
    input memory_to_reg,
    input ALUSrc,
    input reg_write,
    input regdst,
    input [3:0] ALUop,
    output [5:0] upcode
    );

    wire [4:0] o_mux1;
    wire [31:0] o_mux2,o_mux3,o_mux4,o_mux5;
    wire [31:0] pc,pc4,pc_b,pc_j,Ex,Ex2,instr,RF1,RF2,o_ALU,o_Mem,w;
    wire [3:0] op;
    wire [27:0] w1;
    wire Zero,PCSrc;

   PC U1 (clk,reset,o_mux5,pc);

   instruction U2 (pc,instr);
   assign upcode = instr[31:26];

   MUX1 U3 (instr[20:16],instr[15:11],regdst,o_mux1);

   Register_file U4 (clk,instr[25:21],instr[20:16],o_mux1,o_mux3,RF1,RF2,reg_write);

   sign_extend U5 (instr[15:0],Ex);

   MUX2 U6 (RF2,Ex,ALUSrc,o_mux2);

   ALU_control U7 (ALUop,instr[5:0],op);

   ALU U8 (RF1,o_mux2,op,o_ALU,Zero);

   data_memory U9 (o_ALU,RF2,clk,memory_read,memory_write,o_Mem);

   MUX2 U10 (o_ALU,o_Mem,memory_to_reg,o_mux3);

   ADD U11 (pc,32'd4,pc4);

   shift_left2 U12 ({6'b0,instr[25:0]},w);
   assign w1 = w[27:0];
   assign pc_j = ({pc[31:28],w1}); //??

   shift_left2 U13 (Ex,Ex2);

   ADD U14 (pc4,Ex2,pc_b);

   assign PCSrc = branch & (Zero ^ instr[26]);
   MUX2 U15 (pc4,pc_b,PCSrc,o_mux4);

   MUX2 U16 (o_mux4,pc_j,jump,o_mux5);


endmodule
```

کد ۱۲-۴: ماژول datapath

در پیاده‌سازی انجام‌شده، Datapath به صورت یک ماژول سطح بالا طراحی شده که تمامی زیرماژول‌های پردازنده را در خود نمونه‌سازی می‌کند. این ماژول داده‌ها را میان واحدهای مختلف جابه‌جا کرده و مسیر اجرای دستورها را بر اساس سیگنال‌های کنترلی تعیین می‌کند.

جریان اجرای دستور در این ساختار به صورت زیر است:

۱. ابتدا مقدار موجود در Program Counter به حافظه دستور ارسال می‌شود.
۲. دستور fetch شده به طور هم‌زمان در اختیار Control Unit، Register File، Sign Extend و ALU Control قرار می‌گیرد.
۳. Register File دو عملوند موردنیاز را می‌خواند.
٤. در صورت نیاز، مقدار Immediate توسط Sign Extend به ۳۲ بیت گسترش می‌یابد.
٥. مالتی‌پلکسر ورودی ALU تعیین می‌کند که عملوند دوم از رجیستر خوانده شود یا از مقدارImmediate
٦. واحد ALU عملیات موردنظر را انجام می‌دهد.
۷. اگر دستور از نوع LW یا SW باشد، خروجی ALU به عنوان آدرس حافظه استفاده می‌شود.
۸. در مرحله Write Back، مالتی‌پلکسر مربوطه مشخص می‌کند که نتیجه ALU یا داده خوانده‌شده از حافظه در فایل ثبات ذخیره شود.
۹. به طور هم‌زمان، مسیر محاسبه PC+4، آدرس Branch و آدرس Jump فعال است و در نهایت مالتی‌پلکسر آخر مقدار جدید Program Counter را انتخاب می‌کند

نکته ای که در خروجی PCSrc وجود دارد گذاشتن یک گیت XOR برخلاف شماتیک اولیه از پردازنده است. این تغییر برای اضافه نکردن سیگنال خروجی اضافی از control unit طراحی شد. با توجه به اینکه opcode دو دستور BNQ و BEQ تنها در بیت [instr[26 تفاوت دارند (BEQ=000100 و BNQ=000101 )، این بیت را با Zero flag بدست آمده از ماژول ALU ، XOR می کنیم تا اگر Zero flag=1 و بیت داده شده ۰ بود دستور BEQ و اگر Zero flag=0 و بیت داده شده ۱ بود عمل BNQ انجام گیرد.

## ۱۲-۴- ماژول Control Unit :

عملکرد این ماژول در فصل ۳ توضیح داده شد و نقش دقیق هر یک از سیگنال‌های کنترلی و تأثیر آن‌ها بر مسیر اجرای دستورها در فصل پنجم بررسی خواهد شد. در اینجا تنها کد آن را بررسی می کنیم:

```
module control_unit(
    input clk,
    input reset,
    input [5:0] upcode, //instruction[31-26]
    output reg [3:0] ALUop,
    output reg jump,
    output reg branch,
    output reg memory_read,
    output reg memory_write,
    output reg memory_to_reg,
    output reg ALUSrc,
    output reg reg_write,
    output reg regdst
    );

      always @(*)
    begin
        case(upcode)
        6'b000000 : begin ALUop = 4'b0000; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=0; reg_write=1; regdst=1; end // R-type
        6'b100011 : begin ALUop = 4'b0001; jump=0; branch=0; memory_read=1; memory_write=0; memory_to_reg=1; ALUSrc=1; reg_write=1; regdst=0; end // LW
        6'b101011 : begin ALUop = 4'b0010; jump=0; branch=0; memory_read=0; memory_write=1; memory_to_reg=0; ALUSrc=1; reg_write=0; regdst=0; end // SW
        6'b000100 : begin ALUop = 4'b0011; jump=0; branch=1; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=0; reg_write=0; regdst=0; end // BEQ
        6'b000101 : begin ALUop = 4'b0100; jump=0; branch=1; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=0; reg_write=0; regdst=0; end // BNQ
        6'b000010 : begin ALUop = 4'b0101; jump=1; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=0; reg_write=0; regdst=0; end // JUMP
        6'b001000 : begin ALUop = 4'b0110; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=1; reg_write=1; regdst=0; end // ADDI
        6'b001100 : begin ALUop = 4'b0111; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=1; reg_write=1; regdst=0; end // ANDI
        6'b001101 : begin ALUop = 4'b1000; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=1; reg_write=1; regdst=0; end // ORI
        6'b001010 : begin ALUop = 4'b1001; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=1; reg_write=1; regdst=0; end // SLTI
        default :   begin ALUop = 4'b0001; jump=0; branch=0; memory_read=0; memory_write=0; memory_to_reg=0; ALUSrc=0; reg_write=0; regdst=0; end
        endcase

      end

endmodule
```

کد ۱۳-۴: ماژول control_unit

ر این پروژه، واحد کنترل بر اساس مقدار Opcode موجود در دستور، نوع عملیات را تشخیص داده و سیگنال‌هایی مانند **RegDst**، **ALUSrc**، **MemRead**، **MemWrite**، **MemtoReg**، **RegWrite**، **Branch**، **Jump** و **ALUOp** را تولید می‌کند. این سیگنال‌ها مستقیماً به Datapath ارسال شده و نحوه عملکرد واحدهای مختلف را مشخص می‌کنند.

منطق تصمیم‌گیری این ماژول با استفاده از case پیاده‌سازی شده است و برای هر دستور، مجموعه مشخصی از سیگنال‌های کنترلی تولید می‌شود. این روش علاوه بر سادگی، توسعه سیستم و اضافه کردن دستورهای جدید را نیز آسان می‌کند.

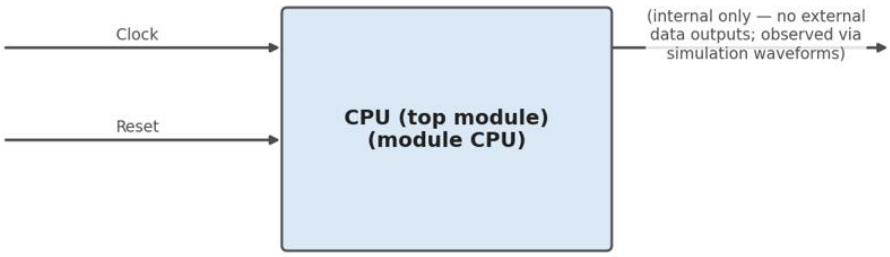
## ۱۳-۴- ماژول CPU :

ماژول CPU بالاترین سطح طراحی در این پروژه است و با ترکیب دو ماژول اصلی Datapath وControl Unit ، ساختار کامل پردازنده را تشکیل می‌دهد. این ماژول تنها شامل دو ورودی Clock و Reset است و تمامی فرآیند اجرای دستورها در داخل آن انجام می‌شود.

```
module CPU(
    input Clock,
    input Reset
    );

   wire jump ,branch ,memory_read ,memory_write ,memory_to_reg ,ALUSrc ,reg_write ,regdst;
   wire [3:0] ALUop;
   wire [5:0] upcode;

   datapath F1 (Clock,Reset,jump,branch,memory_read,memory_write,memory_to_reg,ALUSrc,reg_write,regdst,ALUop,upcode);

   control_unit F2 (Clock,Reset,upcode,ALUop,jump,branch,memory_read,memory_write,memory_to_reg,ALUSrc,reg_write,regdst);

endmodule
```

کد ۱۴-۴: ماژول CPU

از آنجا که CPU تنها نقش یکپارچه‌سازی اجزای پردازنده را بر عهده دارد، پیچیدگی اصلی طراحی در داخل دو ماژول Datapath و Control Unit قرار دارد و خود این ماژول ساختاری نسبتاً ساده دارد.




شکل ۱۲-۴: ماژول CPU

# ۵- تحلیل و ارزیابی مسیر اجرای هر دستور :

پس از طراحی و پیاده‌سازی تمامی واحدهای تشکیل‌دهنده پردازنده، مرحله بعدی بررسی نحوه عملکرد این واحدها در کنار یکدیگر و ارزیابی صحت اجرای دستورها است. اگرچه در فصل چهارم ساختار داخلی هر ماژول به صورت جداگانه مورد بررسی قرار گرفت، اما عملکرد صحیح پردازنده تنها زمانی قابل ارزیابی خواهد بود که ارتباط میان این ماژول‌ها در هنگام اجرای دستورهای مختلف تحلیل شود.

در پردازنده طراحی‌شده، تمامی دستورها از یک مسیر داده مشترک استفاده می‌کنند؛ با این تفاوت که بسته به نوع دستور، تنها بخشی از واحدهای موجود در Datapath فعال می‌شوند. این انتخاب توسط واحد کنترل و از طریق سیگنال‌های کنترلی انجام می‌شود. بنابراین، اگرچه ساختار سخت‌افزاری پردازنده برای تمامی دستورها یکسان است، اما مسیر عبور داده و عملیات انجام‌شده در هر بخش متناسب با نوع دستور تغییر می‌کند.

در ادامه، شرح مسیر اجرای هر یک از دستور ها در سطح RTL گفته می شود؛ یعنی دقیقاً مطابق نام سیم‌ها و سیگنال‌های موجود در کد verilog که در بخش قبل ارائه شد.

## ۵-۱- مسیر اجرای دستورات R-type:

در این گروه از دستورها، هر دو عملوند از فایل ثبات خوانده شده و نتیجه عملیات مجدداً در یکی از ثبات‌ها ذخیره می‌شود. از آنجا که تمامی عملیات در داخل ALU انجام می‌شود، این دستورها نیازی به دسترسی به حافظه داده ندارند و تنها از مسیرهای مرتبط با فایل ثبات و واحد حسابی و منطقی استفاده می‌کنند.

در MIPS طراحی‌شده، دستورهای ADD، SUB، AND، OR و SLT در این گروه قرار می‌گیرند. اگرچه نوع عملیات این دستورها با یکدیگر متفاوت است، اما تمامی آن‌ها مسیر یکسانی را در Datapath طی می‌کنند و تنها کد عملیاتی ارسال‌شده به ALU تغییر می‌کند.

پس از fetch دستور، Opcode برابر 000000 توسط واحد کنترل تشخیص داده می‌شود. واحد کنترل با توجه به این مقدار، سیگنال‌هایی را تولید می‌کند که مسیر مناسب برای اجرای دستورهای نوع R را در Datapath فعال می‌کنند.

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0000 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |

جدول ۱-۵: سیگنال های کنترلی R-type

اجرای دستور های R-type در ۷ مرحله انجام می شود که به شرح زیر است:

۱. در ابتدای اجرای دستور، مقدار موجود در ثبات Program Counter به Instruction Memory ارسال می‌شود. Instruction Memory با استفاده از این آدرس، دستور متناظر را بازیابی کرده و همزمان جمع‌کننده اول مقدار PC+4 را محاسبه می‌کند تا در صورت عدم وقوع jump یا branch، آدرس دستور بعدی آماده باشد.
۲. Register File آدرس های rs (instr[25:21]) و rt (instr[20:16]) را می گیرد تا داده‌های موجود در این دو ثبات خوانده شوند. (RF1 و RF2)
۳. چون RegDst=1، شماره‌ی رجیستر مقصد (write_addr) از فیلد rd (instr[15:11]) توسط مالتی پلکس انتخاب می‌شود.
۴. چون ALUSrc=0، مالتی پلکس ورودی دوم ALU را برابر RF2 قرار می دهد (نه مقدار ثابت Ex)
۵. در این مرحله، دو داده خروجی Register File وارد ALU می‌شوند. همزمان، فیلد Function موجود در دستور به واحد ALU Control ارسال شده و نوع عملیات موردنیاز تعیین می‌شود. خروجی ALU Control به صورت یک کد سه‌بیتی به ALU ارسال شده و عملیات مناسب اجرا می‌شود. (op)
۶. از آنجا که دستورهای نوع R نیازی به خواندن یا نوشتن داده در حافظه ندارند (RegWrite=1)، مالتی پلکس نتیجه‌ی ALU (چون MemtoReg=0) را انتخاب و آن در رجیستر rd نوشته می‌شود و حافظه‌ی داده اصلاً درگیر نمی‌شود. (فعال بودن سیگنال **RegWrite** باعث می‌شود نتیجه عملیات در فایل ثبات نوشته شود و پردازنده آماده اجرای دستور بعدی گردد)
۷. چون Branch=0 و Jump=0، مقدار بعدی PC برابر PC+4 خواهد بود.

## ۲-۵- مسیر اجرای دستورات Immediate:

دستورهای Immediate گروهی از دستورها در معماری MIPS هستند که در آن‌ها یکی از عملوندها به صورت مقدار ثابت (Immediate) در خود دستور ذخیره شده است. برخلاف دستورهای نوع R که هر دو ورودی را از فایل ثبات دریافت می‌کنند، در این دسته یکی از ورودی‌های ALU مستقیماً از داخل دستور تأمین می‌شود.

در پردازنده طراحی‌شده، دستورهای ADDI، ANDI، ORI و SLTI در این گروه قرار می‌گیرند. هدف اصلی این دستورها انجام عملیات حسابی یا منطقی بین یک ثبات و یک مقدار ثابت است.

در این گروه از دستورها، واحد کنترل با توجه به Opcode ، سیگنال‌هایی تولید می‌کند که باعث می‌شود ورودی دوم ALU از مقدار Immediate تأمین شود. همچنین واحد Sign Extend فعال شده و مقدار ۱۶ بیتی Immediate به ۳۲ بیت توسعه داده می‌شود.

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| متغیر | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 |

جدول ۲-۵: سیگنال های کنترلی immediate

اجرای دستور های Immediate در ۷ مرحله انجام می شود که به شرح زیر است:

۱. در ابتدای اجرای دستور، مقدار موجود در ثبات Program Counter به Instruction Memory ارسال می‌شود. Instruction Memory با استفاده از این آدرس، دستور متناظر را بازیابی کرده و همزمان جمع‌کننده اول مقدار PC+4 را محاسبه می‌کند تا در صورت عدم وقوع jump یا branch، آدرس دستور بعدی آماده باشد.
۲. Register File آدرس های rs (instr[25:21]) و rt (instr[20:16]) را می گیرد تا داده‌های موجود در این دو ثبات خوانده شوند. (RF1 و RF2)
۳. چون RegDst=0، شماره‌ی رجیستر مقصد (write_addr) از فیلد rd (instr[20:16]) توسط مالتی پلکس انتخاب می‌شود.
۴. چون ALUSrc=1، مالتی پلکس ورودی دوم ALU را برابر خروجی sign extend قرار می دهد

۵. در این مرحله، ALU محاسبات مربوطه را با توجه به عملیات انتخاب شده توسط ALU Control انجام می دهد.

۶. از آنجا که دستورهای نوع R نیازی به خواندن یا نوشتن داده در حافظه ندارند (RegWrite=1)، مالتی پلکس نتیجه‌ی ALU (چون MemtoReg=0) را انتخاب و آن در رجیستر rd نوشته می‌شود و حافظه‌ی داده اصلاً درگیر نمی‌شود. (فعال بودن سیگنال **RegWrite** باعث می‌شود نتیجه عملیات در فایل ثبات نوشته شود و پردازنده آماده اجرای دستور بعدی گردد)

۷. چون Branch=0 و Jump=0، مقدار بعدی PC برابر PC+4 خواهد بود.

## ۳-۵- مسیر اجرای دستورات LW و SW:

دستور های Load و Store گروهی از دستور های معماری MIPS هستند که برای برقراری ارتباط میان پردازنده و Data Memory استفاده می شوند. این دستور ها نقش اصلی در انتقال داده بین Register File و Data Memory را دارند. در این گروه : ۱) دستور LW (Load Word) داده را از حافظه خوانده و در یک ثبات ذخیره می کند.

۲) دستور SW (Store Word) داده را از ثبات گرفته و در حافظه ذخیره می کند.

ویژگی مهم این دستورها این است که آدرس حافظه توسط ALU محاسبه می شود، به این صورت که مقدار یک ثبات با مقدار Immediate جمع شده و آدرس موثر تولید می کند.

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0001 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |

جدول ۳-۵: سیگنال های کنترل LW

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0001 | 0 | 0 | 1 | 0 | 0 | X | 1 | X |

جدول ۴-۵: سیگنال های کنترل SW

اجرای دستور های LW و SW در ۶ مرحله انجام می شود که به شرح زیر است:

۱. در ابتدای اجرای دستور، مقدار موجود در ثبات Program Counter به Instruction Memory ارسال می‌شود. Instruction Memory با استفاده از این آدرس، دستور متناظر را بازیابی کرده و هم‌زمان جمع‌کننده اول مقدار PC+4 را محاسبه می‌کند تا در صورت عدم وقوع jump یا branch، آدرس دستور بعدی آماده باشد.

۲. Register File آدرس های (rs (instr[25:21]) و (rt (instr[20:16]) را می گیرد تا داده‌های موجود در این دو ثبات خوانده شوند. (RF1 و RF2)

۳. فیلد آفست ۱۶ بیتی گسترش علامت داده می‌شود (Ex) و چون ALUSrc=1، مالتی پلکس O_Mux2 ورودی دوم ALU را برابر آن قرار می دهد.

۴. ALU عملیات add انجام می‌دهد (ALUop=0010) تا آدرس حافظه (rs+offset) محاسبه شود.

۵. این مرحله برای دو دستور LW و SW متفاوت است. در دستور SW ، مقدار RF2 (رجیستر rt) به‌عنوان write_data به Data Memory فرستاده می‌شود؛ چون MemWrite=1 ، این مقدار در آدرس محاسبه‌شده نوشته می‌شود و چون RegWrite=0، هیچ نوشتنی در Register File رخ نمی‌دهد. در مقابل در دستور LW، نتیجه‌ی ALU (rs+offset) به‌عنوان آدرس به Data Memory فرستاده می‌شود؛ چون MemRead=1 ، مقدار آن خانه خوانده می‌شود. چون MemtoReg=1، مقدار خوانده‌شده از حافظه (نه خروجی ALU) در ورودی write_data رجیستر فایل قرار می‌گیرد. چون RegDst=0 ، مالتی پلکس رجیستر مقصد را فیلد (rt (instr[20:16]) انتخاب می‌کند و چون RegWrite=1 ، نتیجه در آن نوشته می‌شود.

۶. چون Branch=0 و Jump=0، مقدار بعدی PC برابر PC+4 خواهد بود.

## ۴-۵- مسیر اجرای دستورات Branch (BNQ و BEQ):

دستورهای Branch برای تغییر مسیر اجرای برنامه بر اساس یک شرط منطقی استفاده می‌شوند. برخلاف دستورهای حسابی و دسترسی به حافظه که تنها روی داده‌ها عملیات انجام می‌دهند، این دسته از دستورها در صورت برقرار بودن شرط تعیین‌شده، مقدار شمارنده برنامه (Program Counter) را تغییر داده و پردازنده را به آدرس دیگری هدایت می‌کنند.

در پردازنده طراحی‌شده، دو دستور BEQ (Branch if Equal) و BNQ (Branch if Not Equal) پیاده‌سازی شده‌اند. دستور BEQ زمانی Branch را انجام می‌دهد که مقدار دو ثبات برابر باشد، در حالی که BNQ تنها زمانی مسیر اجرای برنامه را تغییر می‌دهد که دو مقدار با یکدیگر برابر نباشند!

در هر دو دستور، مقصد انشعاب از جمع مقدار PC+4 و مقدار Immediate توسعه‌یافته پس از انتقال دو بیتی به چپ محاسبه می‌شود.

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0011 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

جدول ۵-۵: سیگنال های کنترلی BEQ

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0100 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

جدول ۶-۵: سیگنال های کنترلی BNQ

اجرای دستور های BEQ و BNQ در ۶ مرحله انجام می شود که به شرح زیر است:

۱. در ابتدای اجرای دستور، مقدار موجود در ثبات Program Counter به Instruction Memory ارسال می‌شود. Instruction Memory با استفاده از این آدرس، دستور متناظر را بازیابی کرده و همزمان جمع‌کننده اول مقدار PC+4 را محاسبه می‌کند تا در صورت برقرار نبودن branch، آدرس دستور بعدی آماده باشد.

۲. پس از دریافت دستور، واحد کنترل Opcode را بررسی کرده و نوع دستور را تشخیص می‌دهد. سپس رجیستر فایل آدرس های rs (instr[25:21]) و rt (instr[20:16]) را می گیرد تا داده‌های موجود در این دو ثبات خوانده شوند. (RF1 و RF2)

۳. در این مرحله، ALU دو مقدار خوانده‌شده از فایل ثبات را با یکدیگر مقایسه می‌کند. در طراحی حاضر، این مقایسه از طریق انجام عملیات تفریق صورت می‌گیرد. اگر نتیجه تفریق برابر صفر باشد، خروجی Zero فعال می‌شود که نشان‌دهنده برابری دو مقدار است.

۴. در ادامه، مقدار Immediate توسعه‌یافته توسط واحد Shift Left 2 دو بیت به سمت چپ منتقل شده و سپس با مقدار PC+4 جمع می‌شود تا آدرس مقصد انشعاب تولید گردد.
۵. پس از تولید سیگنال Zero ، Datapath تصمیم می گیرد که آیا مقدار Program Counter تغییر کند یا خیر. در این پروژه، این تصمیم با استفاده از رابطه : PCSrc = Branch & (Zero ^ Instr[26]) انجام می گیرد. چون با توجه به اینکه opcode دو دستور BNQ و BEQ تنها در بیت instr[26] تفاوت دارند (BEQ=000100 و BNQ=000101 )، در نتیجه : ۱) در دستور BEQ، زمانی که Zero برابر یک باشد (یعنی دو رجیستر برابر بوده و حاصل تفریقشان صفر باشد)، شرط برقرار بوده و PCSrc=1 و مقدار بعدی PC برابر pc_b می‌شود، ۲) در دستور BNQ، زمانی که Zero برابر صفر باشد (یعنی دو رجیستر برابر نبوده و حاصل تفریقشان صفر نشود)، شرط برقرار بوده و PCSrc=1 و مقدار بعدی PC برابر pc_b می‌شود.
۶. از آنجا که دستورهای Branch هیچ مقداری را در فایل ثبات ذخیره نمی‌کنند، RegWrite=0 و هیچ رجیستری تغییر نمی‌کند و Data Memory نیز درگیر نمی‌شود.

## ۵-۵- مسیر اجرای دستور Jump :

دستور Jump یکی از دستورهای کنترل جریان در معماری MIPS است که برای انتقال بدون شرط اجرای برنامه به یک آدرس مشخص استفاده می‌شود. برخلاف دستورهای Branch که ابتدا نتیجه یک مقایسه را بررسی می‌کنند، دستور Jump بدون وابستگی به هیچ شرطی، مقدار جدیدی را برای شمارنده برنامه (Program Counter) تعیین می‌کند.

در MIPS طراحی‌شده، آدرس مقصد از فیلد ۲۶ بیتی موجود در دستور استخراج می‌شود. این مقدار پس از انتقال دو بیت به سمت چپ، با چهار بیت پرارزش مقدار فعلی Program Counter ترکیب شده و آدرس ۳۲ بیتی مقصد تشکیل می‌شود. سپس این آدرس مستقیماً به ورودی Program Counter ارسال شده و اجرای برنامه از محل جدید ادامه پیدا می‌کند.

| ALUop | Jump | Branch | MemWrite | MemRead | RegWrite | MemtoReg | ALUSrc | RegDst |
|---|---|---|---|---|---|---|---|---|
| 0101 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

جدول ۷-۵: سیگنال های کنترلی JUMP

اجرای دستور JUMP در ۵ مرحله انجام می شود که به شرح زیر است:

۱. مانند سایر دستورها، ابتدا مقدار Program Counter به حافظه دستور ارسال شده و دستور Jump ، fetch می‌شود. هم‌زمان، مقدار PC+4 نیز محاسبه می‌شود.

۲. پس از دریافت دستور، واحد کنترل Opcode را بررسی کرده و دستور Jump را تشخیص می‌دهد. سپس سیگنال Jump فعال می‌شود تا Datapath مسیر مناسب را برای به‌روزرسانی Program Counter انتخاب کند .

۳. در این مرحله، فیلد ۲۶ بیتی آدرس از دستور استخراج شده و وارد واحد Shift Left 2 می‌شود تا دو بیت به سمت چپ انتقال یابد. سپس چهار بیت پرارزش مقدار فعلی Program Counter به ابتدای این مقدار افزوده شده و آدرس کامل مقصد تشکیل می‌شود. در طراحی حاضر، این عملیات توسط قطعه کد: assign pc_j = {pc[31:28],w1} که در آن w1 خروجی واحد Shift Left 2 بوده و چهار بیت پرارزش Program Counter به ابتدای آن متصل می‌شوند.

۴. پس از تشکیل آدرس مقصد، مالتی‌پلکسر نهایی (یعنی O_Mux5) با فعال بودن سیگنال Jump، این آدرس را انتخاب کرده و در لبه بعدی کلاک، مقدار جدید در ثبات Program Counter قرار می‌گیرد. در نتیجه، اجرای برنامه از آدرس مقصد ادامه پیدا می‌کند.

۵. از آنجا که دستور Jump هیچ نتیجه‌ای تولید نمی‌کند و هیچ رجیستر یا حافظه ای در این دستور تغییر نمی کند. (Memwrite=0,Regwrite=0,MemRead=0)

در فصل بعد به ارزیابی پردازنده به کمک testbench و شبیه سازی می پردازیم.

# ۶- ارزیابی عملکرد پردازنده با استفاده از شبیه‌سازی (Testbench):

پس از طراحی، پیاده‌سازی و تحلیل معماری پردازنده در فصل‌های پیشین، در این فصل عملکرد سیستم از طریق شبیه‌سازی مورد ارزیابی قرار می‌گیرد. هدف از این ارزیابی، بررسی صحت عملکرد پردازنده در اجرای دستورهای مختلف و اطمینان از تطابق نتایج حاصل از شبیه‌سازی با عملکرد مورد انتظار است.

برای این منظور، یک فایل text آزمایشی شامل مجموعه‌ای از دستورهای پشتیبانی‌شده توسط پردازنده تهیه شده و در اختیار Instruction Memory قرار داده شده است. این برنامه تمامی گروه‌های اصلی دستورها شامل دستورهای نوع R-type ، دستورهای Immediate ، دستورهای Load و Store ، دستورهای Branch و دستور Jump را پوشش می‌دهد و امکان ارزیابی عملکرد تمامی بخش‌های اصلی پردازنده را فراهم می‌کند.

## ۶-۱- طراحی TestBench :

به‌منظور ارزیابی عملکرد پردازنده، یک Testbench برای تحریک سیستم و اجرای برنامه آزمایشی طراحی شده است. وظیفه Testbench تولید سیگنال‌های اولیه موردنیاز پردازنده، از جمله سیگنال ساعت (Clock) و سیگنال ریست (Reset) ، و فراهم کردن شرایط مناسب برای آغاز اجرای برنامه ذخیره‌شده در حافظه دستور است.

```
module tb;

   reg Clock;
   reg Reset;

   CPU UUU (Clock,Reset);


   initial begin

      Clock = 0;
      Reset = 1;

      #5;

      Reset=0;

   end
      always #10 Clock=~Clock;
endmodule
```

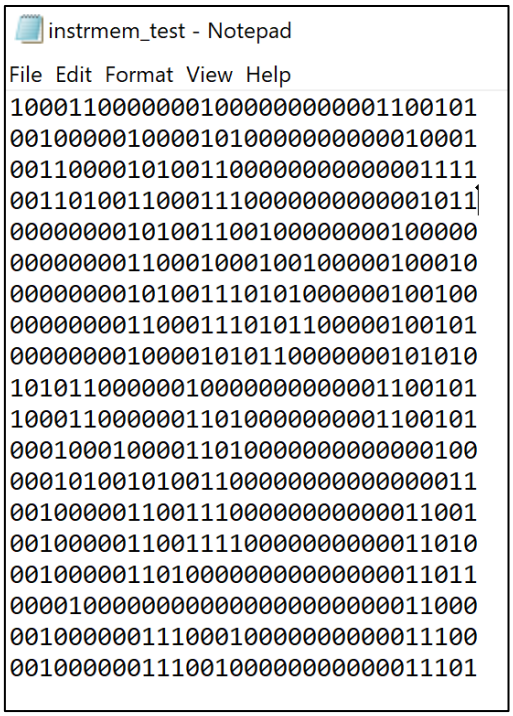
کد ۶-۱: TestBench

در ابتدای شبیه‌سازی، سیگنال ساعت در مقدار صفر و سیگنال Reset در وضعیت فعال قرار می‌گیرد تا تمامی بخش‌های پردازنده در حالت اولیه قرار گیرند. پس از گذشت ۵ نانوثانیه، سیگنال Reset غیرفعال می‌شود تا پردازنده اجرای برنامه را از آدرس صفر آغاز کند. فعال بودن Reset در ابتدای شبیه‌سازی موجب مقداردهی اولیه Program Counter شده و از شروع اجرای پردازنده با مقادیر نامشخص جلوگیری می‌کند

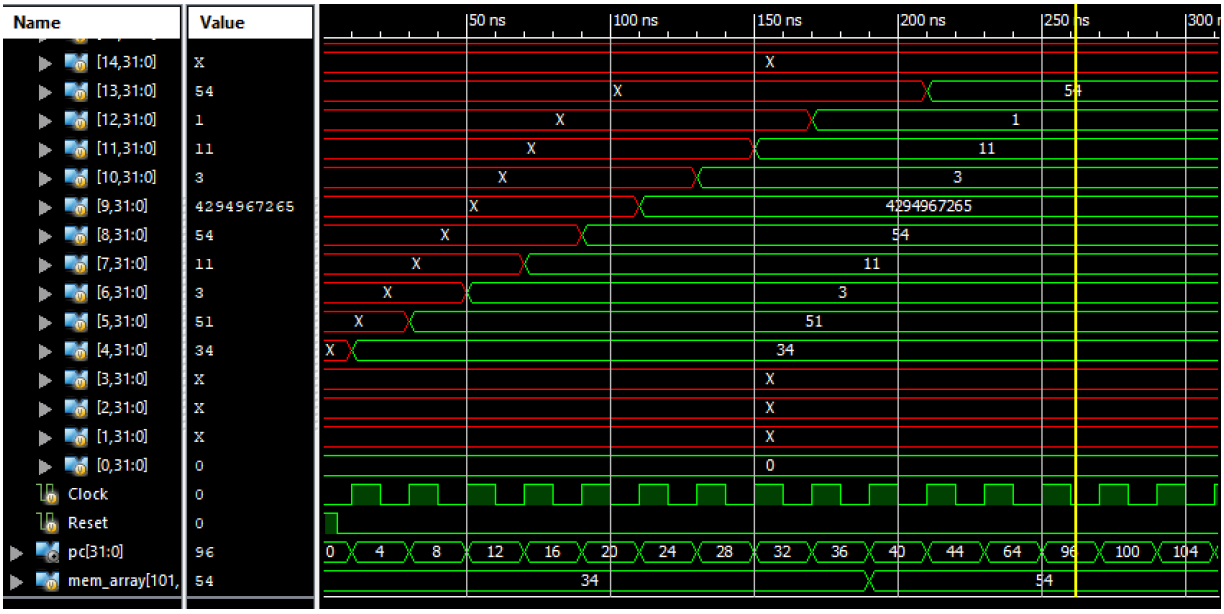
در ادامه با دستور : always #10 Clock = ~Clock ، هر ۱۰ نانوثانیه مقدار Clock تغییر وضعیت می‌دهد؛ بنابراین دوره کامل کلاک برابر ۲۰ نانوثانیه بوده و تمامی تغییرات ترتیبی پردازنده در لبه بالارونده این سیگنال انجام می‌شود.

## ۲-۶- شبیه سازی :

با توجه به دستوراتی که در فایل instrmem_test قرار داده شده و در اختیار Instruction Memory قرار داده ایم، قدم به قدم هر دستور را بررسی و نتیجه آن را ارزیابی می کنیم. همچنین در ابتدا تصویر تمام شکل موج های بدست آمده حاصل از ماژول های PC ، Clock ، Reset و Register File را قرار می دهیم تا نتیجه عملی را با نتیجه تئوری مقایسه نماییم.




شکل ۱-۶: دستورات قرار داده شده در فایل instrmem_test




شکل ۲-۶: شکل موج های خروجی PC ، Clock ، Reset و Register File و mem[101]

## ۱-۲-۶- بررسی دستور اول (دستور LW) :

دستور اول همانطور که در تصویر قرار داده شده، (۱۰۰۰۱۱۰۰۰۰۰۰۰۱۰۰۰۰۰۰۰۰۰۰۰۱۱۰۰۱۰۱) می باشد. ۶ بیت بالای این دستور یعنی ۱۰۰۰۱۱ بیانگر LW بودن این دستور است.

100011 00000 00100 0000000001100101

LW R0 R4 101

در نتیجه با این دستور خواهیم داشت:

R4 = Memory[R0+101] = Memory[101]=34

PC = PC+4 = 4

## ۲-۲-۶- بررسی دستور دوم (دستور ADDI) :

دستور دوم همانطور که در تصویر قرار داده شده، (۰۰۱۰۰۰۰۰۱۰۰۰۰۱۰۱۰۰۰۰۰۰۰۰۰۰۰۱۰۰۰۱) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۱۰۰۰ بیانگر ADD Immediate بودن این دستور است.

001000 00100 00101 0000000000010001

ADDI R4 R5 17

در نتیجه با این دستور خواهیم داشت:

R5 = R4 + 17 = 51

PC = PC+4 = 8

## ۳-۲-۶- بررسی دستور سوم (دستور ANDI) :

دستور سوم همانطور که در تصویر قرار داده شده، (۰۰۱۱۰۰۰۰۱۰۱۰۰۱۱۰۰۰۰۰۰۰۰۰۰۰۰۱۱۱۱) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۱۱۰۰ بیانگر AND Immediate بودن این دستور است.

001100 00101 00110 0000000000001111

ANDI R5 R6 15

در نتیجه با این دستور خواهیم داشت:

R6 = R5 & 15 = 3

PC = PC+4 = 12

## ۴-۲-۶- بررسی دستور چهارم (دستور ORI) :

دستور چهارم همانطور که در تصویر قرار داده شده، (۰۰۱۱۰۱۰۰۱۱۰۰۰۱۱۱۰۰۰۰۰۰۰۰۰۰۰۰۱۰۱۱) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۱۱۰۱ بیانگر OR Immediate بودن این دستور است.

001101 00110 00111 0000000000001011

ORI R6 R7 11

در نتیجه با این دستور خواهیم داشت:

R7 = R6 | 11 = 11

PC = PC+4 = 16

## ۵-۲-۶- بررسی دستور پنجم (دستور ADD R-type) :

دستور پنجم همانطور که در تصویر قرار داده شده، (۰۰۰۰۰۰۰۰۱۰۱۰۰۱۱۰۰۱۰۰۰۰۰۰۰۰۱۰۰۰۰۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۰۰ بیانگر R-type بودن و ۶ بیت پایین بیانگرADD بودن این دستور است.

000000 00101 00110 01000 00000100000

R-type R5 R6 R8 ADD

در نتیجه با این دستور خواهیم داشت: R8 = R5 + R6 = 54

PC = PC+4 = 20

## ۶-۲-۶- بررسی دستور ششم (دستور SUB R-type) :

دستور ششم همانطور که در تصویر قرار داده شده، (۰۰۰۰۰۰۰۰۰۱۱۰۰۰۱۰۰۰۱۰۰۱۰۰۰۰۰۱۰۰۰۱۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۰۰ بیانگر R-type بودن و ۶ بیت پایین بیانگر SUB بودن این دستور است.

000000 00110 00100 01001 00000100010

R-type R6 R4 R9 SUB

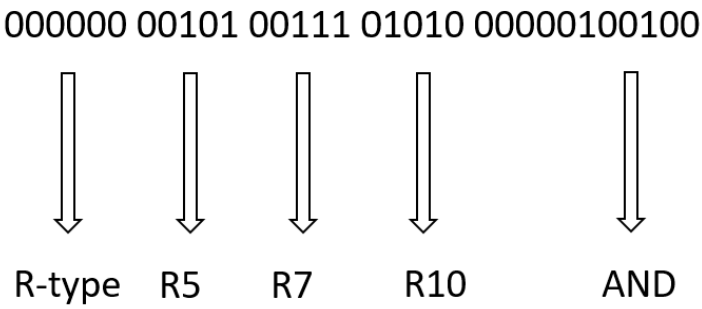
در نتیجه با این دستور خواهیم داشت:

R9 = R6 - R4 = -31

PC = PC+4 = 24

## ۷-۲-۶- بررسی دستور هفتم (دستور AND R-type) :

دستور هفتم همانطور که در تصویر قرار داده شده، (۰۰۰۰۰۰۰۰۰۱۰۱۰۰۱۱۱۰۱۰۱۰۰۰۰۰۰۱۰۰۱۰۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۰۰ بیانگر R-type بودن و ۶ بیت پایین بیانگر AND بودن این دستور است.




در نتیجه با این دستور خواهیم داشت:

R10 = R5 & R7 = 51&11 = 3

PC = PC+4 = 28

## ۸-۲-۶- بررسی دستور هشتم (دستور OR R-type) :

دستور هشتم همانطور که در تصویر قرار داده شده، (۰۰۰۰۰۰۰۰۰۱۱۰۰۰۱۱۱۰۱۰۱۱۰۰۰۰۰۱۰۰۱۰۱) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۰۰ بیانگر R-type بودن و ۶ بیت پایین بیانگر OR بودن این دستور است.

000000 00110 00111 01011 00000100101

R-type R6 R7 R11 OR

در نتیجه با این دستور خواهیم داشت:

R11 = R6 | R7 = 3|11 = 11

PC = PC+4 = 32

## ۹-۲-۶- بررسی دستور نهم (دستور SLT R-type) :

دستور نهم همانطور که در تصویر قرار داده شده، (۰۰۰۰۰۰۰۰۱۰۰۰۰۱۰۱۰۱۱۰۰۰۰۰۰۰۱۰۱۰۱۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۰۰ بیانگر R-type بودن و ۶ بیت پایین بیانگر SLT بودن این دستور است.

000000 00100 00101 01100 00000101010

R-type R4 R5 R12 SLT

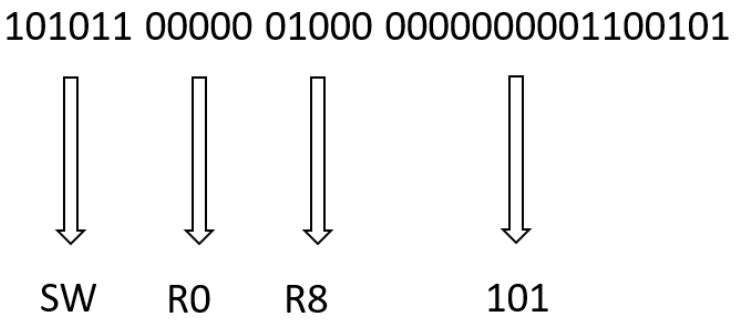
در نتیجه با این دستور خواهیم داشت:

R12 = R4 < R5 = 1

PC = PC+4 = 36

## ۱۰-۲-۶- بررسی دستور دهم (دستور SW) :

دستور دهم همانطور که در تصویر قرار داده شده، (۱۰۱۰۱۱۰۰۰۰۰۰۱۰۰۰۰۰۰۰۰۰۰۰۰۰۱۱۰۰۱۰۱) می باشد. ۶ بیت بالای این دستور یعنی ۱۰۱۰۱۱ بیانگر Store Word بودن این دستور است.




در نتیجه با این دستور خواهیم داشت:

Memory[101] = R8 = 54

PC = PC+4 = 40

## ۱۱-۲-۶- بررسی دستور یازدهم (دستور LW) :

دستور یازدهم همانطور که در تصویر قرار داده شده، (۱۰۰۰۱۱۰۰۰۰۰۰۱۱۰۱۰۰۰۰۰۰۰۰۰۰۱۱۰۰۱۰۱) می باشد. ۶ بیت بالای این دستور یعنی ۱۰۰۰۱۱ بیانگر Load Word بودن این دستور است.

100011 00000 01101 0000000001100101

LW R0 R13 101

در نتیجه با این دستور خواهیم داشت:

R13 = Memory[101] = 54

PC = PC+4 = 44

## ۱۲-۲-۶- بررسی دستور دوازدهم (دستور BEQ) :

دستور دوازدهم همانطور که در تصویر قرار داده شده، (۰۰۰۱۰۰۰۱۰۰۰۰۱۱۰۱۰۰۰۰۰۰۰۰۰۰۰۰۰۱۰۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۱۰۰ بیانگر BEQ بودن این دستور است. در این دستور دو رجیستر R8 و R13 با یکدیگر مقایسه می شوند و در صورت برابر بودن محتوای این دو، PC به ۴+۱۶=۲۰ تا جلوتر می رود.

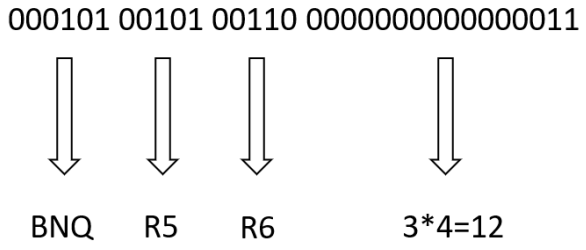
000100 01000 01101 0000000000000100

BEQ R8 R13 16

در نتیجه با این دستور خواهیم داشت: PC = PC+(4+4*4) = 64

## ۱۳-۲-۶- بررسی دستور سیزدهم (دستور BNQ) :

دستور سیزدهم همانطور که در تصویر قرار داده شده، (۰۰۰۱۰۱۰۰۱۰۱۰۰۱۱۰۰۰۰۰۰۰۰۰۰۰۰۰۰۱۱) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۱۰۱ بیانگر BNQ بودن این دستور است. در این دستور دو رجیستر R6 و R5 با یکدیگر مقایسه می شوند و در صورت برابر نبودن محتوای این دو، PC به ۴+۱۲=۱۶ تا جلوتر می رود.




اما این دستور اجرا نمی شود؛ زیرا PC در مرحله قبل تغییر کرد و لذا از این دستور پریدیم.

## ۱۴-۲-۶- بررسی دستورات چهاردهم تا شانزدهم:

دستورات چهاردهم تا شانزدهم که immediate هستند گذاشته شده اند تا اگر دستورات Branch اجرا نشدند، این دستورات اجرا شوند؛ که با توجه به اجرا شدن دستور BEQ ، PC تغییر کرده و از این دستورات پریدیم.

## ۱۵-۲-۶- بررسی دستور هفدهم (دستور JUMP) :

دستور هفدهم همانطور که در تصویر قرار داده شده، (۰۰۰۰۱۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۰۱۱۰۰۰) می باشد. ۶ بیت بالای این دستور یعنی ۰۰۰۰۱۰ بیانگر JUMP بودن این دستور است. با توجه به بیت های پایین این دستور، PC مستقیما و بدون هیچ شرطی به PC=24*4=96 می رود.

000010 00000000000000000000011000

JUMP 24*4=96

در نتیجه با این دستور خواهیم داشت: PC = 4*24 = 96

### ۱۶-۲-۶- بررسی دستورات هجدهم و نوزدهم:

دستورات هجدهم و نوزدهم که immediate هستند گذاشته شده اند تا اگر دستور JUMP اجرا نشد، این دستورات اجرا شوند؛ که با توجه به اجرا شدن دستور JUMP ، PC تغییر کرده و از این دستورات پریدیم.

## ۷- جمع بندی و چالش های طول پروژه:

در این پروژه، یک پردازنده تک‌چرخه‌ای (Single Cycle Processor) بر پایه معماری MIPS با استفاده از Verilog طراحی، پیاده‌سازی و شبیه‌سازی شد. در ابتدا اجزای اصلی پردازنده شامل Program Counter ، Instruction Memory، Register File، Sign Extend، واحد ALU، Data Memory، مالتی‌پلکسرها و واحد کنترل طراحی شدند. سپس با اتصال این ماژول‌ها، مسیر داده و واحد کنترل تشکیل شده و پردازنده کامل ایجاد گردید.

پس از تکمیل طراحی، عملکرد پردازنده با استفاده از برنامه‌ای آزمایشی شامل دستورهای مختلف از جمله Load ، Store، دستورهای محاسباتی، دستورهای Immediate ، دستورهای Branch شرطی و JUMP مورد ارزیابی قرار گرفت. نتایج حاصل از شبیه‌سازی نشان داد که پردازنده قادر است تمامی دستورهای پیاده‌سازی‌شده را مطابق عملکرد مورد انتظار اجرا کرده و

تغییرات صحیح را در شمارنده برنامه (PC) ، Register File و Data Memory اعمال نماید. بنابراین، صحت عملکرد معماری طراحی‌شده در سطح عملکردی مورد تأیید قرار گرفت.

در طول پروژه مشکلات و چالش هایی نیز از قبیل: اطمینان از عملکرد صحیح مسیرهای Branch و Jump و محاسبه درست آدرس مقصد ، هماهنگ‌سازی صحیح سیگنال‌های کنترلی بین Datapath و Control Unit برای هر گروه از دستورها وجود داشت که با موفقیت حل شدند.